SOFTWARE REQUIREMENTS, ARCHITECTURE AND DELIVERY GUIDE

DevTeam Hub

Complete Developer Implementation Blueprint

- Product features in depth - page by page
- System architecture and technology decisions
- Project workflow and delivery process
- Security design and engineering controls
- Data model, API, DevOps, testing and delivery plan

Prepared for: Freelance / contract engineering team

Based on: DevTeam Hub In-Depth Page-by-Page Product Specification (1 November 2026)

This edition: 2 November 2026 | Version 1.0 | Status: Proposed - for product-owner confirmation

Platforms: Web application and mobile application (iOS / Android)

Confidential - for use by the client and the contracted delivery team only.

Contents

How to Use This Document 6

Document control . . . 6
Suggested reading order for a new freelancer . . . 6
Conventions used . . . 6

1. Product Overview 7

1.1 What DevTeam Hub is . . . 7
1.2 Problems it solves . . . 7
1.3 Goals and non-goals . . . 7
1.4 Experience principles (Source) . . . 7
1.5 Personas . . . 8
1.6 Glossary . . . 8

2. Working Agreement for the Delivery Team 9

2.1 Roles on the project . . . 9
2.2 Delivery cadence (Recommended) . . . 9
2.3 Definition of Ready (a work item may be started) . . . 9
2.4 Definition of Done (a work item may be closed) . . . 10
2.5 Source control and review rules . . . 10
2.6 Coding standards (Recommended) . . . 10
2.7 Change control and communication . . . 10

3. Hierarchy, Roles and Access Model 11

3.1 Product hierarchy (Source) . . . 11
3.2 Role presets (Source) . . . 11
3.3 Recommended permission matrix . . . 12
3.4 How access is decided (core algorithm) . . . 12
3.5 Cross-cutting access rules (Source, mandatory) . . . 13

4. System Architecture 14

4.1 Architecture principles . . . 14
4.2 Logical architecture . . . 14
4.3 Technology decisions . . . 15
4.4 Monorepo structure (Source: monorepo) . . . 15
4.5 Backend module design . . . 15
4.6 Multi-tenancy design . . . 16
4.7 Request lifecycle . . . 17
4.8 Background processing . . . 17
4.9 Real-time updates . . . 17
4.10 Caching and performance . . . 17
4.11 File upload architecture . . . 18

4.12 Frontend architecture . . . 18
4.13 Mobile architecture . . . 18
4.14 Environments and deployment topology . . . 18
4.15 Scaling path . . . 19
5. Project Workflow 20
5.1 Build workflow for each feature . . . 20
5.2 Work item lifecycle . . . 20
5.3 Bug workflow (Recommended default) . . . 21
5.4 Code-to-work traceability workflow . . . 21
5.5 Sprint lifecycle . . . 21
5.6 Release workflow . . . 21
5.7 Approval workflow . . . 21
5.8 Notification flow . . . 22
5.9 End-to-end example . . . 22
6. Web Application: Access and Core Workspace Pages 23
6.0 Global application shell . . . 23
6.1 Sign-in page . . . 23
6.2 Accept invitation and account setup . . . 24
6.3 Organization selection and onboarding . . . 24
6.4 Home dashboard . . . 25
6.5 Global search . . . 25
6.6 My Work . . . 26
6.7 Work item detail . . . 27
6.8 Create / edit work item (modal) . . . 27
6.9 Project directory . . . 28
6.10 Project overview and settings . . . 28
6.11 Team and department directories and detail . . . 29
6.12 People directory and employee profile . . . 30
6.13 Calendar and timeline . . . 30
7. Web Application: Agile Delivery, Quality and Collaboration 31
7.1 Board (Kanban / Scrum) . . . 31
7.2 Backlog . . . 31
7.3 Sprint planning, active sprint and sprint report . . . 32
7.4 Bug and defect management . . . 33
7.5 QA test management . . . 33
7.6 Release management and deployment overview . . . 34
7.7 Work log / time tracking . . . 34
7.8 Knowledge base (documentation) . . . 35
7.9 Discussions, comments and mentions . . . 35
8. GitHub Integration and Engineering Activity 37

8.1 Integration architecture . . . 37
8.2 GitHub connections hub . . . 38
8.3 Repository directory and repository detail . . . 39
8.4 Pull request detail and development panel . . . 40
8.5 GitHub activity and contribution page . . . 40

9. Analytics and Performance Insights 42

9.1 Metric catalog (define once, reuse everywhere) . . . 42
9.2 Reports and analytics landing . . . 42
9.3 Organization, department, team and project analytics . . . 43
9.4 Performance insight: principles . . . 43
9.5 Scoring model (Source weights, Recommended mechanics) . . . 44
9.6 Individual performance overview and score detail . . . 45
9.7 Leaderboard (organization / department / team) . . . 46
9.8 Goals and feedback . . . 47

10. Administration, Automation and Integrations 48

10.1 Organization settings . . . 48
10.2 User and access administration . . . 48
10.3 Roles and permissions . . . 49
10.4 Workflow and field configuration . . . 49
10.5 Automation rules . . . 50
10.6 Notifications and inbox . . . 50
10.7 Integrations marketplace and API access . . . 51
10.8 Audit log and security center . . . 51

11. Mobile Application and Shared Interaction Rules 53

11.7 Mobile-specific requirements . . . 53
11.8 Shared interaction, validation and notification rules . . . 54

12. Security Design and Engineering Controls 55

12.1 Security objectives and threat model . . . 55
12.2 Authentication . . . 55
12.3 Authorization enforcement . . . 56
12.4 Tenant isolation checklist . . . 56
12.5 Data protection and cryptography . . . 57
12.6 Secrets management . . . 57
12.7 Application and API security . . . 58
12.8 Integration and webhook security . . . 58
12.9 Protecting performance and personal data . . . 59
12.10 Audit logging . . . 59
12.11 Mobile security . . . 59
12.12 Infrastructure and cloud security . . . 59
12.13 Monitoring, detection and incident response . . . 60

12.14 Privacy and compliance . . . 60
12.15 Secure development lifecycle . . . 60
12.16 OWASP Top 10 coverage map . . . 61
12.17 Security acceptance checklist . . . 61
13. Data Model 62
13.1 Modelling rules . . . 62
13.2 Core tables and key columns . . . 63
13.3 Example DDL (work_items) and indexing . . . 64
13.4 Key relationships . . . 64
14. API Specification Guide 66
14.1 Conventions . . . 66
14.2 Endpoint catalog (representative) . . . 67
15. DevOps, Observability and Operations 68
15.1 CI/CD pipeline . . . 68
15.2 Database migrations and feature flags . . . 68
15.3 Observability . . . 68
15.4 Reliability, backup and disaster recovery . . . 68
16. Quality Assurance and Testing Strategy 69
16.1 Test data and environments . . . 69
16.2 Quality gates for a milestone . . . 69
17. Delivery Plan, Acceptance and Open Decisions 70
17.1 Phases (Source) with indicative effort (Recommended) . . . 70
17.2 Milestone acceptance checklist . . . 71
17.3 Cross-module acceptance criteria (Source) . . . 71
17.4 Risks and mitigations . . . 72
17.5 Open decisions with recommended defaults . . . 72
Appendices 73
Appendix A - Source specification coverage map . . . 73
Appendix B - Pull request checklist . . . 73
Appendix C - Environment variables (names only) . . . 74
Appendix D - Glossary of engineering terms . . . 74

How to Use This Document

This document converts the DevTeam Hub product specification into a complete build guide. It is written for a freelancer or small contract team that has not seen the project before and needs to understand **what** to build, **how the system should be structured**, **how the work should flow** and **how security must be implemented**.

Document control

Item	Detail
Product	DevTeam Hub - company software development management platform (web + mobile)
Source specification	DevTeam Hub In-Depth Page-by-Page Product Specification, prepared 1 November 2026
This edition	Developer Implementation Blueprint v1.0, 2 November 2026
Audience	Freelance full-stack, mobile, DevOps and QA engineers; client product owner
Status	Proposed. Items marked Configurable or Recommended need product-owner confirmation

Suggested reading order for a new freelancer

Step	Read	Why
1	Sections 1-2 (Overview and Working Agreement)	Understand the goal, scope boundaries and how you are expected to work.
2	Section 3 (Hierarchy, roles and access)	Every screen and API depends on the permission model. Learn it first.
3	Sections 4-5 (Architecture and Workflow)	Know the system shape, tenancy design and the lifecycle of work items.
4	Sections 6-11 (Feature specifications)	Detailed behaviour of each page. Read the modules for your assigned phase.
5	Section 12 (Security)	Mandatory engineering controls. Applies to every module, not only auth.
6	Sections 13-17 (Data, API, DevOps, QA, Plan)	Implementation reference, quality gates and milestone acceptance.

Conventions used

- **Source** statements come directly from the client specification (for example page purposes, roles, scoring categories, GitHub behaviour).
- **Recommended** statements are engineering decisions added in this document to make the specification buildable (for example exact algorithms, table designs, library choices). They stay consistent with the stack named in the source (Next.js, React Native, NestJS, PostgreSQL, Redis, BullMQ) and can be changed with product-owner approval.
- **Configurable** means the value must be stored as an organization or project setting, never hard-coded.
- Requirement words: **MUST** = mandatory, **SHOULD** = strongly recommended, **MAY** = optional.
- Work item keys in examples use the format **DTH-123** (project key + sequence number).

> **Important for the freelancer**
> Do not start by building every page. The agreed MVP is: organization + team + project + work item + board + sprint + GitHub connection + basic delivery dashboard + notifications. Quality/release management, detailed analytics and performance rankings come later and performance rankings must only launch after a fairness pilot (see Sections 9 and 17).

1. Product Overview

1.1 What DevTeam Hub is

DevTeam Hub is a **multi-tenant company platform** for planning, assigning, delivering and measuring software work across departments, teams, projects and repositories. It combines Jira-like work tracking with GitHub engineering signals, analytics and fair, evidence-based employee performance insight. Code activity is treated as *context*, never as a substitute for delivery outcomes.

1.2 Problems it solves

- Work, code and releases live in different tools, so nobody sees one version of delivery progress.
- Managers lack trustworthy, explainable data about team flow, quality and capacity.
- Employee performance reviews are often based on opinion or on misleading volume metrics (commit counts, lines of code, hours worked).
- Companies need strict access control: a manager should see only the people and projects inside their scope.

1.3 Goals and non-goals

Goals	Non-goals (out of scope for v1)
Single source of truth for work items, sprints, releases and code activity	Replacing GitHub (no code hosting, no code review UI of its own beyond linked status)
Role- and scope-aware dashboards and analytics	Payroll, billing of end customers, or HR compensation workflows
Transparent, disputable, role-specific performance scores	Automatic employment decisions (promotion, compensation, termination)
Web = full planning/admin; mobile = fast execution and approvals	Complex admin, workflow design and bulk data management on mobile
Auditable, tenant-isolated, least-privilege security model	Non-GitHub Git providers (GitLab/Bitbucket) in v1 - design adapters so they can be added

1.4 Experience principles (Source)

- **Organization-first structure:** one organization contains departments, teams, projects, members, policies and GitHub resources.
- **Work is the central record:** every task, bug, story or epic has owner, status, priority, dates, estimate, relationships, comments and history.
- **Engineering context:** commits, branches, pull requests, reviews, releases and deployments link to work items.
- **Permission-aware analytics:** menu visibility never implies data access; scope is enforced on every query.
- **Fair performance insight:** every score explains its evidence, sample size and model version.
- **Responsive experience:** web is complete; mobile prioritises updates, task execution, approvals and notifications.

1.5 Personas

Persona	Main needs	Primary screens
Developer / Engineer	See my queue, update status fast, link PRs, understand my own performance evidence	My Work, Work item, Board, GitHub activity, Self performance (web + mobile)
Team Lead / Scrum Master	Plan sprints, balance workload, unblock the team, run retrospectives	Backlog, Sprint planning, Active sprint, Team analytics
Project Manager	Track milestones, risks, releases and project health	Project overview, Timeline, Releases, Project analytics
QA Engineer	Triage bugs, run test cycles, verify fixes, keep traceability	Bugs, Test management, Release readiness
DevOps Engineer	Record and monitor deployments, link releases to commits	Deployments, Repository detail, Releases
Department Head / Executive	Portfolio health, capacity, quality and delivery trends without private details	Department and Organization analytics
Organization Admin / Security Admin	Manage people, roles, integrations, policies and audit	Admin pages, Audit log, Security center

1.6 Glossary

Term	Meaning
Organization (tenant)	A company account. All data belongs to exactly one organization.
Scope	The part of the hierarchy a role assignment applies to (organization, department, team, project, repository, personal).
Work item	Generic record for task, story, bug, epic, request or sub-task.
Workflow	Set of statuses and allowed transitions for a work type in a project.
Sprint	Timeboxed iteration with a goal, capacity and committed work.
Snapshot	Immutable, point-in-time computed result (analytics or performance) kept for reproducibility.
Evidence	A concrete linked record (work item, PR, review, defect) that justifies a metric value.
Identity mapping	Verified link between a GitHub account and an employee.
ACL	Access control list restricting a single resource (document, comment, attachment).
RLS	PostgreSQL Row-Level Security: database-enforced tenant filtering.

2. Working Agreement for the Delivery Team

This section defines how a freelancer should work so that code from different contributors is consistent, reviewable and shippable. Adapt names and cadences to your contract, but keep the intent.

2.1 Roles on the project

Role	Responsibility
Product owner (client)	Owns scope, confirms open decisions, accepts milestones, provides representative users for UX validation.
Tech lead / architect	Owns architecture decisions, reviews critical PRs, maintains the decision log (ADRs). May be the senior freelancer.
Backend engineer(s)	NestJS modules, database migrations, queues, GitHub sync, security controls, API tests.
Frontend engineer(s)	Next.js web app, design system integration, permission-aware UI, accessibility.
Mobile engineer	React Native app, secure storage, push notifications, offline-safe queue.
QA engineer	Test plans, automation, tenant-isolation and permission tests, release verification.
UX/UI designer	Flows, wireframes, design system, validation copy, empty/error states.
DevOps engineer	Infrastructure as code, CI/CD, observability, backups.

A single freelancer may cover several roles; the responsibilities still apply and the client should be told which hat is being worn when a decision is made.

2.2 Delivery cadence (Recommended)

- Two-week sprints. Sprint planning on day 1, daily asynchronous status note, mid-sprint check-in, demo and retrospective on the last day.
- Each milestone ends with a **working demo on a staging environment**, release notes and a completed acceptance checklist (Section 17).
- Weekly written progress report: completed items, blockers, risks, next-week plan, open questions for the product owner.
- Open decisions (Section 17.5) must be raised **before** building the affected module, not after.

2.3 Definition of Ready (a work item may be started)

- Clear user story and acceptance criteria referencing the relevant page in this document.
- Permissions and scope behaviour stated (who can see/do what).
- Design or wireframe available for UI work; API contract drafted for integration work.
- Dependencies identified; estimate agreed.

2.4 Definition of Done (a work item may be closed)

Area	Done means
Code	Implemented, linted, type-checked (TypeScript strict), no TODOs without linked work item, reviewed by at least one other engineer or the tech lead.
Tests	Unit tests for business rules, integration tests for API and DB, permission tests (allowed and denied), cross-tenant test for any new data path.
Security	Server-side authorization on every endpoint, input validation, no secrets in code or logs, audit event emitted for sensitive actions.
UX	Loading, empty, error, no-permission and offline (mobile) states implemented; keyboard accessible; non-colour status cues.
Data	Migration is reversible or has a documented rollback; indexes for new filters; seed data updated.
Docs	API documented in OpenAPI; README/ADR updated if behaviour or architecture changed.
Delivery	Merged via PR, CI green, deployed to staging, verified against acceptance criteria.

2.5 Source control and review rules

- Monorepo with trunk-based development: short-lived branches from **main**, merged by pull request. Protect main (required reviews, required status checks, no force-push).
- Branch naming: **feature/DTH-123-short-title**, **fix/DTH-456-short-title**, **chore/...**. Include the work item key so the platform can auto-link the branch.
- Commit messages follow Conventional Commits: **feat(work-items): add transition validator**. PR title includes the key.
- Keep PRs small (target under 400 changed lines excluding generated files). Large changes are split behind feature flags.
- PR template checklist: linked work item, screenshots for UI, migration notes, security impact, test evidence, rollback plan.
- Review turnaround target: 1 business day. Reviewer checks correctness, permission handling, tenant isolation, performance (N+1 queries) and readability.
- Never commit secrets. Use environment variables and a secret manager; run secret scanning in CI.

2.6 Coding standards (Recommended)

- TypeScript strict mode everywhere; no implicit any; shared types in a **packages/shared** library; runtime validation with Zod (web/mobile) and class-validator or Zod pipes (API).
- Backend: controllers are thin; business rules live in services/domain classes; data access only in repositories; modules communicate through exported services or domain events, never through each other's tables.
- Frontend: server components for data-heavy pages, TanStack Query for client data, React Hook Form + Zod for forms, a permission hook (**useCan**) for conditional UI. UI hiding is cosmetic only.
- Naming: snake_case in database, camelCase in code and JSON, kebab-case in URLs, PascalCase for classes and components.
- All dates stored as UTC timestamptz; user timezone applied only at presentation. Money is not handled in v1.
- Errors use the standard problem format (Section 14). Never leak stack traces or internal IDs of inaccessible resources.

2.7 Change control and communication

- Any change that alters scope, a permission rule, a scoring weight or a data retention rule requires written approval from the product owner (log it as an ADR or change request).
- Record architecture decisions as short ADRs in **/docs/adr** (context, decision, alternatives, consequences).
- Raise risks early: integration limits (GitHub rate limits), data volume, unclear requirements.
- Hand-over package at the end: source, infrastructure code, runbooks, environment variable list, test reports, admin guide.

3. Hierarchy, Roles and Access Model

3.1 Product hierarchy (Source)

Level	Represents	Typical owner
Organization	Company tenant, policies, membership, configuration	Organization owner / admin
Department	Business or engineering division	Department head / admin
Team	People collaborating on a delivery stream	Team lead
Project	Outcome, product, initiative or delivery scope	Project manager / lead
Board / Sprint	Operational view and timeboxed planning	Team / project roles
Work item	Task, story, bug, epic, request, sub-task	Assignee / reporter
GitHub resources	Repositories, PRs, reviews, releases, deployments	Connected account / installation

Open decision
The source leaves open whether a user can belong to multiple departments or teams. Recommended: allow many-to-many team membership (a person can be in several teams) and a single primary department. The schema in Section 13 supports both.

3.2 Role presets (Source)

Role	Typical capabilities	Default scope
Organization Owner	Full tenant control, ownership transfer, billing, all settings	Organization
Organization Admin	Members, teams, projects, integrations, policies, reports	Organization
Department Head	View/manage department teams, projects, capacity, analytics	Assigned department
Team Lead	Team membership, boards, sprint planning, team reporting	Assigned team
Project Manager	Project config, work, milestones, reports, delivery	Assigned project
Technical Lead	Technical planning, review visibility, repo mapping, engineering reports	Assigned teams/projects/repos
Developer / Engineer	Create/update permitted work, link branches/PRs, comment, log work	Assigned projects/items
QA Engineer	Test/bug workflows, verification, defect triage, QA reporting	Assigned projects
DevOps Engineer	Release/deployment records and relevant repo/workflow data	Assigned projects/repos
Employee / Viewer	View allowed work and analytics; limited editing	Granted scope

Organizations may create **custom roles**. Custom roles are only collections of permissions; they can never bypass the permission checks described below.

3.3 Recommended permission matrix

Legend: **M** = manage (create, edit, delete, configure), **W** = create/edit within scope, **R** = read within scope, **-** = no access. Every letter applies only inside the role's assigned scope. Start from this matrix and let the product owner adjust it.

Resource	Own	Adm	DH	TL	PM	Tech	Dev	QA	Ops	View
Organization settings and policies	M	M	-	-	-	-	-	-	-	-
Users, invitations, roles	M	M	-	-	-	-	-	-	-	-
People directory	R	R	R	R	R	R	R	R	R	R
Departments	M	M	M	R	R	R	R	R	R	R
Teams	M	M	M	M	R	R	R	R	R	R
Projects (create/configure)	M	M	W	W	M	R	R	R	R	R
Work items	M	M	R	M	M	W	W	W	W	R
Boards, backlog, sprints	M	M	R	M	M	W	W	W	R	R
Workflow and field configuration	M	M	-	-	M	-	-	-	-	-
Bugs and QA test management	M	M	R	R	M	R	W	M	R	R
Releases and deployments	M	M	R	R	M	W	R	W	M	R
GitHub org connection	M	M	-	-	R	W	-	-	R	-
Repository mapping	M	M	-	R	W	W	R	-	W	-
Team / project analytics	M	M	R	R	R	R	R	R	R	R*
Organization analytics	M	M	R**	-	-	-	-	-	-	-
Individual performance (others)	M***	M***	R***	R***	R***	-	-	-	-	-
Leaderboard configuration	M	M	-	-	-	-	-	-	-	-
Automation rules	M	M	-	W	W	-	-	-	-	-
Integrations, API keys, webhooks	M	M	-	-	-	R	-	-	W	-
Audit log and security center	M	M	-	-	-	-	-	-	-	-

* only where the viewer was explicitly granted the project/team. ** department scope only. *** only within reporting or team scope and only if the organization policy allows; every user can always see their own performance page.

3.4 How access is decided (core algorithm)

Authorization is **permission + scope + resource policy**. A request is allowed only if all three pass:

- **Permission**: the user holds a role (directly or via a team/department membership) that includes the action, for example **work_item:transition**.
- **Scope**: the role assignment covers the resource. A scope assignment is (role, scope_type, scope_id). A project-scoped role covers that project; a team-scoped role covers all projects mapped to the team; a department-scoped role covers its teams and their projects; an organization-scoped role covers everything in the tenant.
- **Resource policy**: extra restrictions on the resource itself (private project, restricted document ACL, confidential comment, performance visibility setting, field-level restriction).
- **Deny wins**: an explicit deny or a stricter resource ACL overrides inherited allows.

```
function can(user, action, resource):
    ctx = loadTenantContext(user.organizationId)             // org membership must be active
    if not ctx.membershipActive:            return DENY
    grants = ctx.assignmentsFor(user)                        // [(role, scopeType, scopeId)]
    allowed = grants.any(g => g.role.permissions.includes(action)
                           and scopeCovers(g.scopeType, g.scopeId, resource))
    if not allowed:                          return DENY
    if resource.acl and not resource.acl.allows(user):       return DENY   // stricter ACL wins
    if policyBlocks(ctx.orgPolicy, action, resource, user):  return DENY   // e.g. performance visibility
    return ALLOW   // and write an audit event if action is flagged sensitive
```

Implement this once in an **AuthorizationService** and call it from NestJS guards and service methods. List endpoints must apply the same rules as a **query filter** (return only visible rows) so counts and search snippets never leak inaccessible data.

3.5 Cross-cutting access rules (Source, mandatory)

- Every API request verifies: authenticated user, organization membership, requested resource scope and action permission.
- Switching organization changes the tenant context; no data may cross organizations.
- Managers see team/project performance evidence only inside their scope and policy.
- Sensitive actions (role changes, integration secrets, exports, audit access, score adjustments) require elevated permission and audit logging.
- Showing the Analytics menu does not grant access to all employees' private or restricted data.

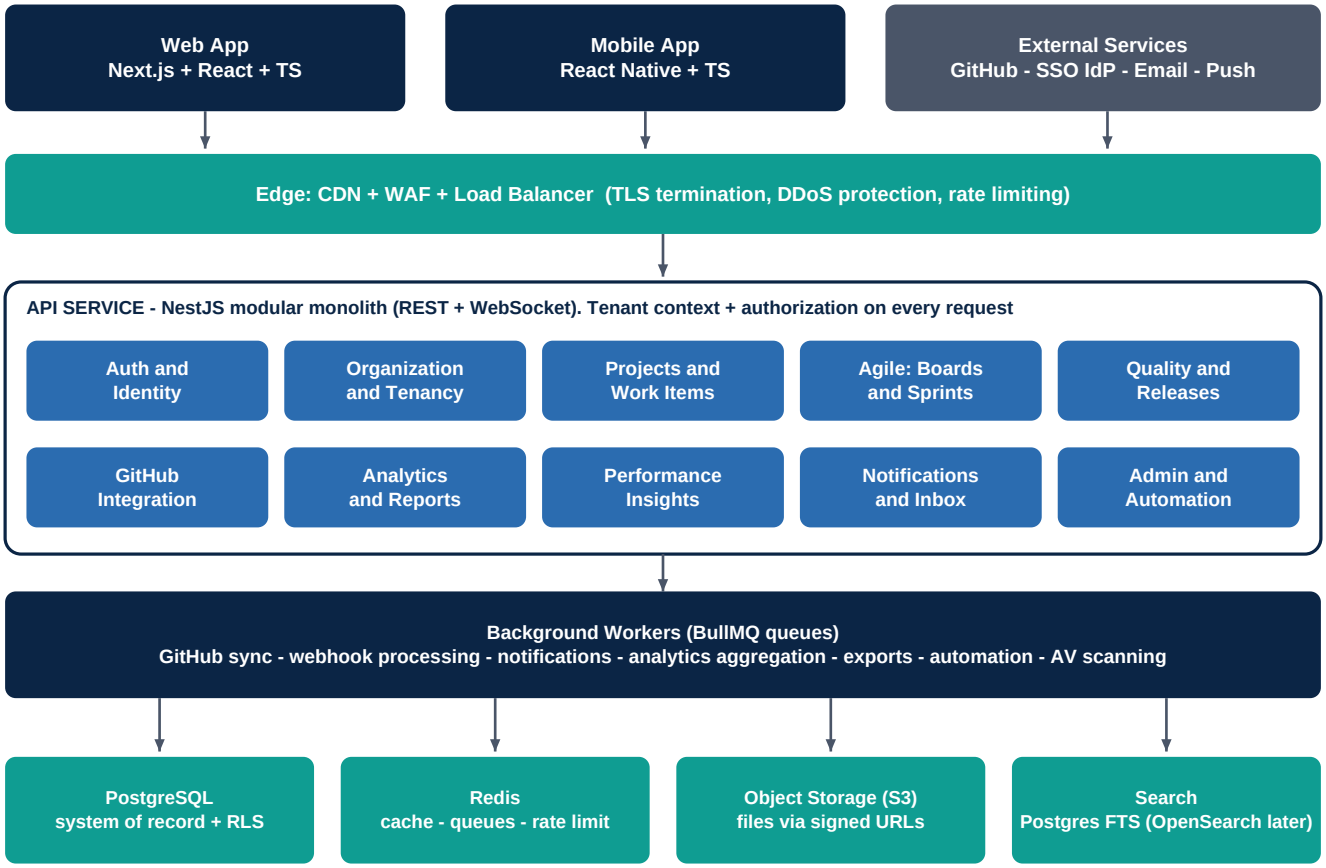
4. System Architecture

This section explains how the system should be built. The architecture follows the source recommendation: a **modular monolith first**, with clear domain boundaries and background workers, split into services only when scale or team ownership justifies it.

4.1 Architecture principles

- **Tenant isolation is non-negotiable.** Every row, query, job, cache key, file path and search document belongs to exactly one organization.
- **Authorization on the server, always.** Hidden buttons are a convenience; the API is the security boundary.
- **Append-oriented history.** Work item history, audit events and score snapshots are never overwritten.
- **Asynchronous by default for integrations.** Webhooks are acknowledged fast and processed by workers; user requests never wait on GitHub.
- **Evidence over opinion.** Analytics and performance are computed from stored, linkable records and versioned definitions.
- **Graceful degradation.** If GitHub, email or search is down, core work tracking continues and the UI shows last-synced status.

4.2 Logical architecture



Cross-cutting: Observability (logs, metrics, traces, alerts) - Secrets manager + KMS - Audit log - CI/CD

Figure 1 - Logical architecture. Clients talk only to the API service through the edge layer. The API owns all business rules; workers handle slow or external work; data stores are never exposed to clients.

4.3 Technology decisions

Layer	Choice (Source / Recommended)	Notes
Web UI	Next.js + React + TypeScript	Responsive. Server-side authorization checks. Tailwind CSS with an accessible component kit (for example shadcn/ui or MUI). TanStack Query, React Hook Form + Zod, dnd-kit for boards, ECharts or Recharts for charts.
Mobile	React Native + TypeScript	Expo or bare workflow (decide in Phase 0). Keychain/Keystore for tokens, encrypted local store for offline queue, FCM/APNs for push.
API	Node.js (current LTS) + NestJS + TypeScript	REST with OpenAPI; WebSocket gateway for selected real-time updates. Guards, interceptors and pipes implement the request lifecycle.
Database	PostgreSQL (system of record)	Prisma or equivalent ORM. Row-Level Security as a second line of defence. Migrations versioned in the repo.
Cache / queues	Redis + BullMQ	Caching, rate limiting, job queues, short-lived locks. Never store the only copy of important data in Redis.
Files	S3-compatible object storage	Pre-signed URLs, malware scanning, access check before every download.
Search	PostgreSQL full-text + trigram first	Move to OpenSearch only if volume or relevance requires it. ACL filtering is mandatory in both.
Cloud	AWS or equivalent managed cloud	Container platform (ECS Fargate or Kubernetes), managed PostgreSQL Multi-AZ, managed Redis, secrets manager + KMS, WAF, CDN.
CI/CD and IaC	GitHub Actions + Terraform	Separate dev / staging / production, automated tests, SAST, dependency and secret scanning.
Observability	OpenTelemetry + metrics/log platform + Sentry	Structured logs with request, tenant and user IDs (never secrets), traces, job and webhook monitoring.

4.4 Monorepo structure (Source: monorepo)

```
devteam-hub/
  apps/
    web/                Next.js web application
    mobile/             React Native application
    api/                NestJS API service (HTTP + WebSocket)
    worker/             BullMQ workers (same domain code, different entrypoint)
  packages/
    shared/             shared types, enums, permission keys, Zod schemas
    api-client/         typed API client generated from OpenAPI
    ui/                 design system components (web)
    config/             eslint, tsconfig, prettier presets
  infra/                Terraform, deployment manifests, runbooks
  docs/                 ADRs, architecture notes, metric definitions, API docs
  .github/workflows/    CI pipelines
```

4.5 Backend module design

Each module owns its tables and exposes a small public service interface. Modules talk through exported services or **domain events** (transactional outbox), never by reading another module's tables directly. This keeps the monolith splittable later.

Module	Responsibilities	Owns (main tables)	Emits events
auth	Login, MFA, SSO, sessions, invitations, password reset	users, sessions, invitations	user.signed_in, user.locked
organizations	Tenants, departments, teams, memberships, settings, policies	organizations, organization_members, departments, teams, team_members	member.added, team.changed
access	Roles, permissions, scope assignments, authorization service, audit writer	roles, permissions, role_permissions, scope_assignments, audit_logs	role.changed

Module	Responsibilities	Owns (main tables)	Emits events
projects	Projects, members, milestones, settings, workflows and fields	projects, project_members, project_teams, milestones	project.created
work-items	Work items, transitions, comments, attachments, history, links, worklogs	work_items, comments, attachments, dependencies, work_item_history, worklogs	work_item.created/updated/transitioned
agile	Boards, backlog ranking, sprints, capacity, retrospectives	boards, board_columns, sprints, sprint_items, sprint_events	sprint.started/completed
quality	Bugs, test cases, runs, results	test_cases, test_runs, test_results	test_run.completed
releases	Releases, approvals, environments, deployments	releases, deployments, environments	release.approved/released
github	OAuth, App installations, repos, webhooks, sync, identity mapping	github_*, repositories, repository_mappings, commits, pull_requests, reviews, workflow_runs	pr.opened/merged, review.submitted
analytics	Metric definitions, snapshots, dashboards, reports, exports	metric_definitions, analytics_snapshots, dashboards, reports	snapshot.ready
performance	Score models, weights, scores, rankings, goals, feedback, adjustments, disputes	performance_*, score_adjustments, ranking_visibility_settings	score.published, dispute.opened
notifications	Event to notification, preferences, delivery channels	notifications, notification_preferences	notification.failed
automation	Rules, executions, loop prevention, integration webhooks and API keys	automation_rules, automation_executions, api_keys, webhooks, webhook_deliveries	automation.executed
knowledge	Documents, versions, templates, ACLs	documents, document_versions	document.published

4.6 Multi-tenancy design

Recommended: a **shared database and schema** with an **organization_id** column on every tenant-owned table, protected twice: (1) application-level tenant scoping in the repository layer and (2) PostgreSQL Row-Level Security so a coding mistake cannot leak data across tenants.

```
-- every tenant table
ALTER TABLE work_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE work_items FORCE ROW LEVEL SECURITY;

CREATE POLICY tenant_isolation ON work_items
  USING      (organization_id = current_setting('app.current_org')::uuid)
  WITH CHECK (organization_id = current_setting('app.current_org')::uuid);

-- per request, inside the same transaction as the queries:
SELECT set_config('app.current_org', $1, true);   -- true = local to transaction
```

- The application database user MUST NOT be a superuser and MUST NOT own the tables (owners bypass RLS unless FORCE is set). Use a separate migration user.
- Wrap each request (and each background job) in a transaction helper that sets **app.current_org** from the verified token, never from a request parameter.
- Composite indexes start with organization_id: for example (organization_id, project_id, status).
- Cross-tenant admin tooling (support) is a separate, audited path with a break-glass procedure.
- Mandatory automated test: authenticate as a user of organization A and try to read, update and delete organization B records through every endpoint. All attempts must return not-found or forbidden.

4.7 Request lifecycle

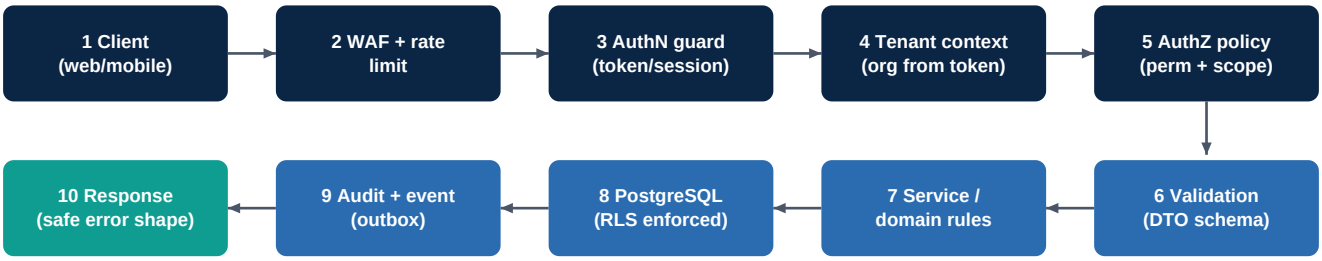


Figure 2 - Lifecycle of every API call. Steps 3 to 6 are NestJS guards/pipes applied globally so a developer cannot forget them; a route must explicitly opt out (for example the sign-in route).

4.8 Background processing

Queue	Job examples	Rules
github-webhooks	Process a stored webhook delivery	Idempotent by delivery ID; retry with exponential backoff; dead-letter queue after N failures; alert admins on sustained failure.
github-sync	Initial repository backfill, scheduled reconciliation, metadata refresh	Respect GitHub rate limits; use conditional requests (ETag); pausable per connection.
notifications	Fan-out, email, push, digests	Check recipient access at send time; collapse duplicates; record delivery status.
analytics	Nightly and incremental snapshots, report generation	Deterministic and re-runnable; write new snapshot version rather than update in place.
performance	Period close, score calculation, ranking snapshot	Runs only after data-completeness checks; stores model version and inputs.
exports	CSV/PDF export of authorized data	Re-check permission when the job runs; signed, expiring download link; audit entry.
automation	Rule executions and scheduled rules	Loop prevention, per-rule rate limits, service identity with explicit permissions.
files	Malware scan, thumbnail	File is unavailable until scan result is clean.

4.9 Real-time updates

- Use WebSockets only where value is clear: board changes, comments on an open work item, notification badge, sprint progress.
- Authenticate on connection; authorize each **subscription** (join room org:{id}:project:{id}) using the same authorization service.
- Send minimal payloads (event type, resource ID, version number). The client refetches through the normal REST endpoint, so permission checks and field filtering are never bypassed.
- Fallback to polling when the socket drops; show a "reconnecting" indicator.

4.10 Caching and performance

- Redis keys always start with the tenant: **org:{orgId}:...**. Never share a key across organizations.
- Cache effective permissions per user for a short TTL (for example 60 seconds) and invalidate on **role.changed** and membership events.
- Cache heavy aggregates (dashboards, analytics) from snapshots, not from live scans of work_items.
- Paginate every list (cursor pagination), index every filter, avoid N+1 queries, and run exports asynchronously.
- Planning targets (to validate): common reads p95 under 500 ms, writes p95 under 800 ms, dashboard first meaningful content under 3 s (excluding external providers).

4.11 File upload architecture

- Client asks the API for an upload slot, sending file name, size and declared type. The API checks permission on the parent resource, size and an allow-list of types.
- API returns a short-lived pre-signed URL (single object, size limit, content-type bound) pointing to a **quarantine** bucket/prefix.
- After upload, a worker scans the file with an antivirus engine and verifies the real content type (do not trust the extension). On success it moves the object to the clean location and marks the attachment **available**; on failure it is deleted and the uploader is notified.
- Downloads go through the API: permission check, then a short-lived signed URL. Serve user files from a separate domain with **Content-Disposition: attachment** for non-images to avoid script execution.

4.12 Frontend architecture

- App Router with route groups: (auth), (app) for the authenticated shell, and per-module folders matching the page names in this document.
- The shell (left nav, top bar, organization switcher, global create, right detail panel) is a shared layout. Changing organization clears all client caches.
- Use a **permissions provider** that receives the effective permissions of the user from **GET /me**; components call **useCan("work_item:create", scope)**. The server still enforces everything.
- Filters and view state live in the URL query string so views are shareable (never put secrets or private data in URLs).
- Standard components for loading skeletons, empty states, error boundaries, permission-denied and conflict dialogs (see Section 11.8).
- Accessibility is part of the component library: focus management, ARIA labels, contrast, non-colour status indicators, keyboard drag-and-drop alternatives on the board.

4.13 Mobile architecture

- Shares types, Zod schemas and the API client with the web app through the monorepo packages.
- Tokens in Keychain/Keystore only. Local cache stores the minimum data for recently viewed work and is wiped on logout and on organization switch.
- Offline queue: edits are saved as pending operations with an idempotency key, shown as **queued** until the server confirms; conflicts show server version vs local change.
- Push notifications carry only an event ID and tenant/resource reference; the app fetches content after authentication so no sensitive text sits on the lock screen unless the user opts in.

4.14 Environments and deployment topology

Environment	Purpose	Data
Local	Developer machine with Docker Compose (Postgres, Redis, mock S3, mail catcher)	Seed/fake data only
Dev	Continuous integration target, feature previews	Synthetic data
Staging	Production-like. Milestone demos, UAT, performance tests, security tests	Anonymised or synthetic. Never real employee data
Production	Live tenants. Multi-AZ database, backups, alerting, WAF	Real data, encrypted, access restricted

Production reference: CDN + WAF in front of a load balancer; API and worker containers in private subnets across at least two availability zones; PostgreSQL and Redis in isolated data subnets with no public access; outbound traffic through NAT with allow-listed destinations where practical; separate AWS accounts (or equivalent) for staging and production.

4.15 Scaling path

- Scale API and workers horizontally (stateless). Scale queues independently per workload.
- Add PostgreSQL read replicas for analytics reads; partition high-volume tables (github_events, work_item_history, audit_logs) by time or tenant when needed.
- Extract a service (likely GitHub ingestion or analytics) only when it has a clear scaling or ownership reason; the module and event boundaries make this possible.

5. Project Workflow

This section covers two things: how the **delivery team builds** DevTeam Hub, and how **work flows inside the product** once built. Understanding the second is essential because most business rules are about transitions, links and snapshots.

5.1 Build workflow for each feature

Step	Activity	Output
1	Read the page specification and acceptance criteria; list permissions and states	Understanding checklist
2	Raise open questions with the product owner; update the decision log	Confirmed rules
3	Design the API contract (OpenAPI) and data changes; review with tech lead	Contract + migration plan
4	Build migration, repository, service, controller, permission rules and audit events	Backend slice
5	Write unit, integration and permission/tenant tests	Green test suite
6	Build UI with all states (loading, empty, error, denied, conflict, offline for mobile)	Frontend slice
7	Self-review against Definition of Done and security checklist	PR ready
8	Peer review, CI, deploy to staging	Merged and deployed
9	Demo against acceptance criteria; product owner accepts or lists changes	Accepted work item

5.2 Work item lifecycle

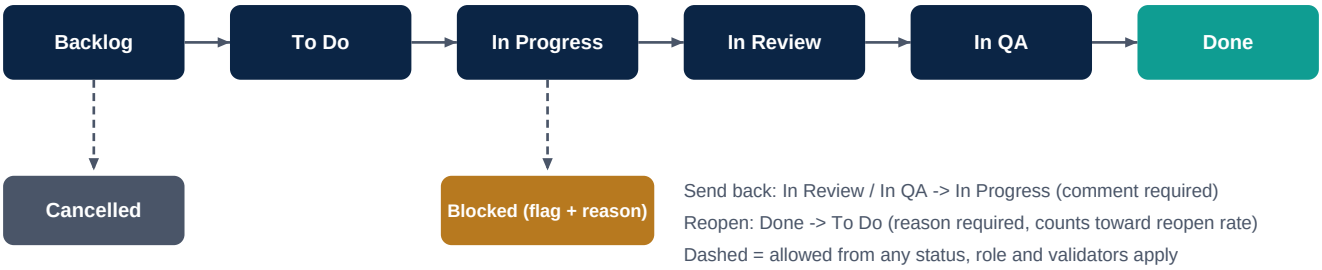


Figure 3 - Default work item workflow. Workflows are configurable per project and work type; this is the recommended starting template.

Transition	Who	Validators / required data
Backlog to To Do	Team lead, PM, developer with sprint rights	Item ranked; estimate present (warning if missing); added to a sprint or board.
To Do to In Progress	Assignee, team lead	Assignee required (auto-assign actor if empty); start date recorded.
In Progress to In Review	Assignee	Optional per project: linked branch or PR required; checklist complete.
In Review to In QA	Reviewer, tech lead	Optional: PR approved and checks passing; review comments resolved.
In QA to Done	QA, team lead	Verification result recorded; resolution set; open sub-tasks must be closed.
Back to In Progress	Reviewer, QA	Comment required; counts as rework for quality metrics.
Done to To Do (reopen)	QA, lead, PM	Reason required; reopen count increments (feeds quality score).
Any to Cancelled	Lead, PM	Resolution (Will not do / Duplicate / Obsolete) and reason.

Transition	Who	Validators / required data
Block / unblock (flag)	Assignee, lead	Reason required; optional link to blocking item; blocked duration is measured.

Rule
If a transition is invalid or the validators fail, the API returns a clear error and the UI keeps the card in its original column. The system must never silently change a status. Every transition writes a history record: who, when, from, to, comment.

5.3 Bug workflow (Recommended default)

New - Triaged - In Progress - In Review - Ready for QA - Verified - Closed. Side exits: **Duplicate** (links to master bug), **Cannot Reproduce** (needs reporter reply), **Reopened** (regression, reason required). Required bug fields: severity, priority, environment, affected version, steps to reproduce, expected and actual results. Closing requires a QA verification record.

5.4 Code-to-work traceability workflow

- Developer creates a branch containing the work item key, for example **feature/DTH-123-board-wip-limits**.
- Commits and the PR title mention the key. The GitHub integration parses keys from branch names, PR titles/descriptions and commit messages and links them automatically; developers can also link manually.
- PR opened - work item shows the PR in its development panel. Optional automation: move item to In Review.
- Checks and reviews update the panel in near real time through webhooks.
- PR merged - optional automation moves the item to In QA (or Done if the project has no QA step). A merged PR with no linked work item is flagged when traceability policy is on.
- Deployment events attach the commit to a release; release notes list the linked work items.

5.5 Sprint lifecycle

Stage	What happens	Rules
Planned	Create sprint, set name, goal, dates, team, capacity. Add items from backlog.	Dates cannot overlap another active sprint for the same team (configurable). Warn on missing estimates, blockers, over-capacity.
Active	Team executes. Board, burndown/burnup, blockers, scope change log.	Adding or removing items requires a reason and is logged. A start snapshot of committed scope is stored.
Completed	Lead completes sprint. Incomplete items are carried to next sprint or backlog.	End snapshot stored. Sprint report is generated and retrospective opens.
Cancelled	Sprint stopped before completion.	Items return to backlog; reason recorded; excluded from predictability trends.

5.6 Release workflow

Draft - Planned - Readiness review - Approved - Released. Exit states: Withdrawn, Rolled back. Readiness checklist (configurable): all scope items Done, no open critical/high bugs, required test runs passed, required approvals received, release notes drafted, deployment plan linked. A release blocked by a critical defect cannot be approved until the defect is resolved or an authorized user records a documented exception.

5.7 Approval workflow

- Approval requests can be attached to work items or releases. Approvers are users or roles; decision is approve or reject with an optional comment (comment required on reject).
- Approvals can expire; expired requests show a clear state and can be re-requested. Approvers act from web or mobile inbox.

- Permission is re-checked when the decision is submitted. Decisions are audit-logged and immutable.

5.8 Notification flow

- Domain event is written in the same database transaction as the change (transactional outbox) so events are never lost.
- Notification worker resolves recipients (assignee, watchers, mentioned users, approvers, reviewers), applies user preferences and digest settings.
- Before delivery, the worker verifies the recipient can still access the target resource. Content is generic if access was revoked.
- Channels: in-app (WebSocket + inbox), push, email, digest. Critical security notices cannot be muted when policy requires.
- Delivery status is stored; failures retry, then surface as failed delivery in admin monitoring.

5.9 End-to-end example

#	Action	System behaviour
1	Team lead creates story DTH-120 "Add WIP limits" in the backlog, estimate 5	Work item stored; history entry; watchers set; event emitted.
2	Sprint planning adds DTH-120 to Sprint 14	Capacity bar updates; sprint_items row; scope change not logged (sprint not started).
3	Developer moves card To Do - In Progress on the board	Transition validator passes; assignee set; board updates for others in real time.
4	Developer pushes branch feature/DTH-120-wip-limits and opens PR	Webhook received, verified, queued, processed; PR linked to DTH-120; card shows PR badge.
5	Reviewer approves; checks pass; PR merged	Review and merge events stored as evidence; automation moves item to In QA; QA gets notification.
6	QA runs test cases, records pass, moves item to Done	Test result linked; resolution set; sprint burndown updates.
7	Release manager adds DTH-120 to release 2.4; deployment webhook arrives	Release readiness updates; deployment linked to commit and release; release notes draft includes DTH-120.
8	Sprint ends; snapshot and report are generated	Metrics (cycle time, review latency, completion) feed analytics and, with other evidence, performance snapshots.

6. Web Application: Access and Core Workspace Pages

Each page below follows the same structure: purpose, users, access rules, screen layout, behaviour, business rules, data and API, states, and acceptance criteria. Pages may be merged into tabs or panels during UX design, but every listed capability must stay discoverable.

6.0 Global application shell

- **Left navigation:** Home, My Work, Inbox, Organizations/Departments/Teams (as permitted), Projects, Boards, Sprints, Calendar, GitHub, Reports and Analytics, Performance, Knowledge Base, Automations, Integrations, Settings. Items the user cannot use are hidden.
- **Top bar:** organization switcher, global search, Create button, help, notification bell, user menu.
- **Context header:** breadcrumb, title, description, primary action, view/filter controls.
- **Content area:** responsive; saved filters; consistent loading, empty and error states.
- **Right-side detail panel:** quick-view of a work item, notification or activity without losing the list position.
- **Global create:** work item, bug, project (if authorized), document, announcement.
- **Organization switch:** triggers a full context reset: clear client caches, reconnect WebSocket, reload effective permissions.

6.1 Sign-in page

Purpose	Authenticate employees and administrators into the correct organization.
Primary users	All users.
Access rules	Unauthenticated page. Organization authentication policy (password, SSO, MFA) applies.

Screen layout and components

- Brand mark; email/username; password with show/hide; remember-device only if policy permits.
- Buttons: Continue with Google / Microsoft / SSO if configured. GitHub sign-in only if explicitly enabled for identity.
- Links: forgot password, invitation acceptance, privacy and terms.

User actions and behaviour

- Validate credentials; enforce MFA and SSO policy; show generic errors.
- If the user belongs to several organizations, route to organization selection; otherwise to onboarding or dashboard.
- Record sign-in events (success, failure, MFA, lockout) for security review.

Business rules

- Never reveal whether an email exists (same message and similar response time for unknown user and wrong password).
- Throttle by IP and by account; temporary lock after repeated failures; notify the user on lock.
- SSO-enforced organizations disable password login for their members.

Data and API

- POST /api/v1/auth/login, /auth/mfa/verify, /auth/refresh, /auth/logout; GET /api/v1/me.
- Tables: users, sessions, organization_members, audit_logs (auth events).

States and edge cases

- Invalid credentials, locked account, expired password, MFA required, SSO unavailable, suspended membership.

Acceptance criteria

- Wrong password and unknown email produce identical responses.

- MFA-required users cannot obtain a session before completing MFA.
- Suspended members cannot enter the organization.

6.2 Accept invitation and account setup

Purpose	Let an invited person securely join an organization and complete their profile.
Primary users	Invited employee; organization admin (sender).
Access rules	Only the valid invite recipient (or an approved domain/SSO flow) can accept.

Screen layout and components

- Invitation context: company, inviter, assigned role/team (if disclosed), expiry.
- Form: name, password or SSO setup, optional avatar, timezone, notification preferences, terms and privacy acknowledgement.

User actions and behaviour

- Accept or decline; set credentials; complete MFA enrolment if required; join organization; go to first-run onboarding.
- Expired or revoked invitations offer a request-new-invitation action.

Business rules

- Invitation token is random, single-use, stored hashed, expires (for example 7 days, configurable).
- Email on the invitation must match the authenticated or created account unless admin allowed domain-based join.
- Role and team/project grants in the invitation are applied atomically on acceptance and audit-logged.

Data and API

- POST /api/v1/invitations (admin), GET /api/v1/invitations/:token, POST /api/v1/invitations/:token/accept.
- Tables: invitations, users, organization_members, scope_assignments.

States and edge cases

- Expired, already used, revoked, mismatched email, organization disabled.

Acceptance criteria

- A used or expired token cannot be reused.
- Accepting creates membership with exactly the invited role and scope.

6.3 Organization selection and onboarding

Purpose	Help users select a tenant and establish their initial working context.
Primary users	New users and users in multiple organizations.
Access rules	Only memberships are selectable. Organization creation depends on the tenant provisioning policy.

Screen layout and components

- Organization cards: name, logo, user role, recent access.
- First-run checklist: profile, team, project, notification settings, GitHub connection.
- Admin setup path: organization profile, departments, teams, members, roles, integrations; progress bar and next recommended step.

User actions and behaviour

- Select organization; resume incomplete onboarding; invite colleagues; create first project.

Business rules

- Onboarding progress is stored per user and per organization; steps are skippable except mandatory security steps.
- Selecting an organization sets the active tenant context for the session.

Data and API

- GET /api/v1/organizations, POST /api/v1/organizations/:id/select, GET/PATCH /api/v1/onboarding.

States and edge cases

- No organizations, suspended tenant, incomplete setup, pending invitation.

Acceptance criteria

- A user sees only organizations they belong to.
- Suspended tenants show a clear message and block entry.

6.4 Home dashboard

Purpose	Provide a role-aware snapshot of work, delivery and team activity.
Primary users	All signed-in users.
Access rules	Each widget enforces its own data scope independently. A shared dashboard layout never exposes confidential data.

Screen layout and components

- Greeting, organization context, date-range selector.
- My open work, overdue work, assigned bugs, upcoming deadlines, recent updates.
- Team/project widgets: sprint progress, blocked items, recently updated work, upcoming releases.
- Quick actions: create task, open board, log work, connect GitHub (if permitted). Widget reorder, hide/show, saved layouts.

User actions and behaviour

- Open a work item from a widget; change the time range; drill into a report; filter by team, project, status, date.
- Customize and save layout; create additional dashboards if permitted.

Business rules

- Widgets load independently so one failure never blanks the page.
- Aggregates come from cached snapshots; show a "last updated" time.
- Default layout depends on role (developer, lead, manager) and is overridable per user.

Data and API

- GET /api/v1/dashboards/home, GET/PUT /api/v1/dashboards/:id/layout, widget endpoints per data source.
- Tables: dashboards, work_items, sprints, notifications, releases, analytics_snapshots.

States and edge cases

- New organization, no assigned tasks, GitHub disconnected, stale analytics, widget load failure.

Acceptance criteria

- A user with access to only one project sees only that project in every widget.
- A failed widget shows retry without affecting others.

6.5 Global search

Purpose	Find work, projects, people, teams, documents, repositories, pull requests and releases quickly.
Primary users	All users.
Access rules	Only authorized resources appear. Snippets and result counts must not leak inaccessible items.

Screen layout and components

- Search box with recent searches and suggestions; result tabs: All, Work, Projects, People, Teams, Documents, Repositories, Pull Requests, Releases.
- Filters: organization, project, status, assignee, type, updated date. Highlighted snippets with metadata.

User actions and behaviour

- Debounced search as you type; submit for full results; save search; open result; clear history if policy allows.

Business rules

- ACL filter is applied inside the query (join to visible projects/teams), not after the fact.
- Minimum query length (for example 2 characters). Support exact key lookup (DTH-123) with top priority.
- Indexing is eventual; show "indexing in progress" when applicable.

Data and API

- GET /api/v1/search?q=&type;=&filters;=; GET/POST /api/v1/saved-searches.
- Index: tsvector + GIN on work_items, documents, projects, people; trigram on keys and names.

States and edge cases

- No results, partial indexing, search service unavailable, too-short query.

Acceptance criteria

- A user cannot find a private project or its work items, even by exact key.
- Counts equal only the visible results.

6.6 My Work

Purpose	A focused personal queue for every employee.
Primary users	All employees (managers use team views separately).
Access rules	Users see only items they can browse; edits depend on project and workflow permission.

Screen layout and components

- Tabs: Assigned to me, Reported by me, Watching, Recently viewed, Completed. List/board switch.
- Columns: key, title, status, priority, due date, project, estimate, updated. Filters: status, priority, due date, project, type.
- Bulk selection and saved personal filters.

User actions and behaviour

- Open detail; change status; reassign if allowed; set due date; comment; bulk edit; watch/unwatch; export if permitted; sort and group.

Business rules

- Overdue and blocked items are visually flagged with icon plus text (not colour only).
- Items with unresolved dependencies show a warning.
- Bulk edits run per item; show per-item success or failure.

Data and API

- GET /api/v1/work-items?assignee=me&status;=...; PATCH /api/v1/work-items/bulk.
- Tables: work_items, work_item_assignees, watchers, projects.

States and edge cases

- Empty queue, no results, blocked and overdue items, unresolved dependencies, permission loss mid-session.

Acceptance criteria

- Only my assigned items appear in Assigned tab.
- Bulk edit partially failing reports exactly which items failed and why.

6.7 Work item detail

Purpose	A complete, auditable record of a task, story, bug, epic or sub-task.
Primary users	Assignee, reporter, project members, leads, authorized viewers.
Access rules	Browse/edit/transition permissions per project. Private comments and restricted attachments follow their own ACL.

Screen layout and components

- Header: key, summary, type, status, priority, project, quick actions.
- Description editor: rich text, links, checklists, embedded references.
- Fields: assignee(s), reporter, labels, components, sprint, estimate, due date, start date, environment, resolution.
- Sections: activity, comments, history, work log, attachments, linked items, subtasks, GitHub development, tests, approvals.
- Side panel: relationships, parent/child, blockers, watchers, people.

User actions and behaviour

- Edit fields inline; transition via workflow buttons; comment; attach; log time; create subtask; link/block/duplicate/relate; add watchers; link branch/commit/PR; request review or approval.

Business rules

- Every significant field change is timestamped and attributable (history).
- Optimistic concurrency: updates send the item version; if stale the server returns a conflict and the UI shows both versions.
- Required fields on transition (for example resolution) are enforced server-side.
- Deleting is soft delete (archive); hard delete only under retention policy.

Data and API

- GET/PATCH /api/v1/work-items/:id; POST /api/v1/work-items/:id/transitions, /comments, /attachments, /worklogs, /links.
- Tables: work_items, work_item_history, comments, attachments, dependencies, worklogs, pull_requests links.

States and edge cases

- Deleted/archived item, inaccessible linked record (show placeholder without details), invalid transition, upload failure, concurrent edit conflict.

Acceptance criteria

- Every edit produces a history row with actor, old value and new value.
- A linked item the user cannot access is shown only as "restricted item".

6.8 Create / edit work item (modal)

Purpose	Create and edit structured work with minimum friction.
Primary users	Users with create/edit permission.
Access rules	Create/edit permission plus field-level restrictions.

Screen layout and components

- Work type selector; summary; description; project; priority; assignee; due date; estimate; labels.
- Context-aware fields by type and project configuration; optional parent/epic, sprint, component, environment, acceptance criteria, attachments.

User actions and behaviour

- Validate required fields; save draft if supported; create and open; create another; clone; add to sprint; link branch after creation; warn on unsaved changes.

Business rules

- Field schemes decide which fields are visible and required for each project and work type.
- Parent must be allowed by hierarchy (epic > story/task > sub-task).
- Duplicate-submit protection through an idempotency key.

Data and API

- GET /api/v1/projects/:id/create-meta; POST /api/v1/work-items (Idempotency-Key header).

States and edge cases

- Missing required field, invalid date (due before start), unavailable assignee, parent mismatch, duplicate submission.

Acceptance criteria

- Required fields vary by project configuration.
- Double-clicking Create results in one item.

6.9 Project directory

Purpose	Show accessible projects and their status.
Primary users	All users; project creators and managers.
Access rules	Only visible projects are listed; private project metadata is not exposed.

Screen layout and components

- Cards or table: name, key, lead, status, team, progress, dates, last activity. Filters: active/archived, department, team, owner, type. Create project button.

User actions and behaviour

- Open, create from template, favorite, archive (if allowed), search, sort, filter, edit visibility if authorized.

Business rules

- Project key is unique per organization, uppercase letters/digits, 2 to 10 characters, immutable after first work item unless an admin migration runs.
- Project health: On track / At risk / Off track; set by humans with a rationale, supported by computed signals.

Data and API

- GET/POST /api/v1/projects; GET/PATCH /api/v1/projects/:id.

States and edge cases

- No projects, no match for filters, archived project.

Acceptance criteria

- Creating a project requires permission and unique key.
- Archived projects are read-only and hidden by default.

6.10 Project overview and settings

Purpose	Present status, scope, people and delivery signals; configure project behaviour.
Primary users	Project members, PM, team leads, stakeholders; project admins for settings.
Access rules	Project browse permission to view; project admin to manage. Organization mandatory policies override.

Screen layout and components

- Overview: name/key, description, status, owner, dates, health, progress, open/completed/blocked counts, milestone timeline, sprint summary, members, repositories, activity, links and documents.
- Tabs: Overview, Work, Board, Backlog, Sprints, Releases, Reports, Documents, Settings.

- Settings: general, people and roles, team mappings, work types and fields, workflow, notifications, repositories, board defaults, estimates, release settings, archive/delete.

User actions and behaviour

- Edit details; manage members and team access; create milestone; open board/backlog; connect repository; archive or complete; show impact preview before changing key, statuses or fields.

Business rules

- Destructive delete only under retention policy with typed confirmation.
- Changing a workflow status in use requires a migration step mapping old status to new.
- All settings changes are audit-logged.

Data and API

- GET/PATCH /api/v1/projects/:id, /members, /milestones, /settings, /workflows.

States and edge cases

- Project not found, no permission (same response to avoid revealing existence), archived, no work yet, missing repository.

Acceptance criteria

- Impact preview lists affected items before a status mapping change is saved.

6.11 Team and department directories and detail

Purpose	Help employees discover teams/departments and let leads and heads manage them.
Primary users	All members for directories; team leads, department heads, admins for management.
Access rules	Directory visibility is configurable; private team details require scope.

Screen layout and components

- Team directory: name, department, lead, members, active projects, capacity.
- Team detail tabs: Overview, Members, Projects, Board, Sprints, Calendar, Analytics, Settings. Overview shows goals, working agreements, recent work, availability, workload distribution.
- Department directory/detail: head, teams, member count, project portfolio, milestones, capacity, analytics, goals, operating documents.

User actions and behaviour

- Create team/department; invite members; request to join if allowed; assign projects; manage team roles; move teams between departments (policy); open analytics.

Business rules

- A team has exactly one lead (can be changed with audit). A team may be archived; archived teams keep history.
- Moving a team between departments creates a membership snapshot so historical department analytics remain correct.

Data and API

- GET/POST /api/v1/teams, /api/v1/departments; POST /teams/:id/members; PATCH /teams/:id.

States and edge cases

- No teams/departments, empty team, team at capacity, archived team, reassignment pending.

Acceptance criteria

- Moving a team does not rewrite last quarter's department report.

6.12 People directory and employee profile

Purpose	Searchable people directory and a work-focused employee profile.
Primary users	All employees (within visibility policy); managers for their scope.
Access rules	Private fields and performance evidence are hidden by default. Managers need reporting/team scope.

Screen layout and components

- List: name, avatar, title, department, team, timezone, availability if enabled.
- Profile tabs: Overview, Work, Activity, Goals, Feedback, Performance (permission-controlled). Optional skills.

User actions and behaviour

- Search/filter; open profile; mention/message; managers assign work, set goals, give feedback if allowed; users edit own profile and preferences.

Business rules

- Field-level visibility settings per organization (for example hide phone number).
- Deactivated users remain visible on historical items as "Name (inactive)".

Data and API

- GET /api/v1/people, GET /api/v1/people/:id, PATCH /api/v1/me/profile.

States and edge cases

- Deactivated user, external collaborator, no shared project, hidden profile fields.

Acceptance criteria

- A user without a shared project and without directory permission sees only public profile fields.

6.13 Calendar and timeline

Purpose	Show delivery dates, milestones, releases, sprint windows and assigned work in time context.
Primary users	Employees, PMs, leads, stakeholders.
Access rules	Only authorized projects; personal availability details limited by policy.

Screen layout and components

- Month/week calendar and Gantt-style timeline; filters by project, team, person, type.
- Items: milestones, due dates, sprint windows, releases, leave/availability (if enabled), dependencies.

User actions and behaviour

- Create milestone or event; drag to adjust dates with permission; open linked work; detect conflicts and dependency risks; export iCal feed if enabled.

Business rules

- Changing a date that violates a dependency shows a warning with the affected items; the user can accept or cancel.
- All times stored UTC; show user timezone and organization timezone when they differ.

Data and API

- GET /api/v1/calendar?from=&to;=&filters;=; POST /api/v1/milestones; GET /api/v1/calendar/feed/:token (read-only, revocable).

States and edge cases

- Overlapping milestones, unscheduled items, timezone conversion, no events.

Acceptance criteria

- Dragging a dependent task before its blocker warns before saving.

7. Web Application: Agile Delivery, Quality and Collaboration

7.1 Board (Kanban / Scrum)

Purpose	Visualize work by workflow status and help teams move work through delivery.
Primary users	Team members, project members, team leads, project managers.
Access rules	Only visible items. Drag actions check transition permission and workflow validity.

Screen layout and components

- Columns mapped to workflow statuses; cards show key, title, type, priority, assignee, due date and flags.
- Swimlanes by epic, assignee, priority or saved query; WIP limits; quick filters.
- Toolbar: project/team scope, saved filter, search, grouping, display settings, sprint selector (Scrum).
- Card indicators: blocked, overdue, dependency, linked PR, unestimated.

User actions and behaviour

- Drag/drop to a valid status; quick edit; create in column; bulk move; manage columns and WIP limits; switch board view; Scrum boards start/complete sprint if permitted.

Business rules

- Never silently change status: if the transition fails validation the card snaps back and a message explains why.
- WIP limit exceeded shows a warning; hard WIP enforcement is a board setting.
- Ranking within a column uses a lexicographic rank (for example LexoRank-style string) so reordering updates one row.
- Keyboard alternative to drag-and-drop (move card menu) is mandatory for accessibility and mobile.

Data and API

- GET /api/v1/boards/:id; PATCH /api/v1/work-items/:id/rank; POST /api/v1/work-items/:id/transitions.
- Tables: boards, board_columns, board_filters, work_items (rank, status).

States and edge cases

- Transition rejected, WIP warning, offline update, stale board (refresh banner via WebSocket version check), empty column.

Acceptance criteria

- Two users moving cards simultaneously both see consistent final state.
- An invalid drag shows the reason and leaves status unchanged.

7.2 Backlog

Purpose	Prioritize and prepare upcoming work before sprint commitment.
Primary users	Product owner, team lead, PM, team members.
Access rules	Project/team access and edit permission; sprint management restricted to leads.

Screen layout and components

- Backlog list grouped by epic or priority; sprint sections; unscheduled items.
- Fields: rank, key, summary, type, priority, estimate, assignee, status, dependency warning.
- Side panel for selected item; capacity/estimate summary for selected sprint.

User actions and behaviour

- Reorder by rank; move into sprint; create epic/story/task; bulk prioritize, assign, estimate; split work item; show dependencies and unresolved blockers.

Business rules

- Splitting creates new items linked to the original and keeps history; estimates are not duplicated.
- Items already in an active sprint show a lock/notice when moved out (reason required).

Data and API

- GET /api/v1/projects/:id/backlog; PATCH /api/v1/work-items/:id/rank; POST /api/v1/work-items/:id/split.

States and edge cases

- Empty backlog, item already in active sprint, sprint capacity exceeded, rank conflict (server re-ranks and returns new order).

Acceptance criteria

- Reordering persists for all users; bulk actions respect per-item permission.

7.3 Sprint planning, active sprint and sprint report

Purpose	Plan a realistic sprint, track execution, and learn from outcomes.
Primary users	Team lead, Scrum master, PM, team.
Access rules	Sprint create/start/complete permissions for leads; team/project data scope for others.

Screen layout and components

- Planning: name, goal, dates, team, working days, capacity per member, candidate backlog, selected items, estimate totals, dependency/blocked/unestimated warnings.
- Active sprint: goal, remaining days, board/list, burndown/burnup, scope change log, blocked work, completed items, links to stand-up notes and retrospective.
- Report: goal achieved or not, planned vs completed, carryover, scope changes, cycle-time summary, defect trend, retrospective (went well, pain points, experiments with owners).

User actions and behaviour

- Create sprint; add/remove/reorder; set goal and dates; compare commitment to capacity; start; flag blockers; add/remove scope with reason; complete sprint and carry over; create follow-up actions from retrospective as work items; compare sprints; export report.

Business rules

- Capacity = sum over members of (working days minus leave) x availability factor x focus factor (configurable). Compare with committed estimate; warn, do not block.
- On start, store a committed-scope snapshot. On complete, store a final snapshot. Reports read snapshots, not live data.
- Scope added after start is flagged as scope creep; removed items require a reason.
- Reopened work after sprint close is shown as post-close change and does not rewrite the report.

Data and API

- POST /api/v1/sprints, /sprints/:id/start, /sprints/:id/complete; GET /sprints/:id/report; POST /sprints/:id/retrospective.
- Tables: sprints, sprint_items, sprint_events, capacity_allocations, availability.

States and edge cases

- Overlapping sprint, invalid dates, capacity overrun, missing estimates, no team, paused or cancelled sprint, insufficient history.

Acceptance criteria

- Starting a sprint with warnings requires explicit confirmation.

- Completing a sprint moves unfinished items as chosen and produces a stable report.

7.4 Bug and defect management

Purpose	Track defects from discovery through triage, fix, verification and closure.
Primary users	QA, developers, support, team leads, PMs.
Access rules	Project bug access; severity and workflow changes can be limited by role.

Screen layout and components

- Bug list: severity, priority, environment, affected version, reporter, assignee, status.
- Bug detail: steps to reproduce, expected/actual result, logs and screenshots, environment, linked work, duplicates, regression flag, root cause, verification result.

User actions and behaviour

- Create, triage, assign; set severity and priority; link to story, PR or release; move through workflow; reopen with reason; mark duplicate; record verification; generate defect trend reports.

Business rules

- Severity (impact) and priority (urgency) are separate fields with defined scales (for example S1-S4, P1-P4).
- An "escaped defect" is a bug found in production for a released work item; this flag feeds quality metrics.
- Missing reproduction steps keeps a bug in "Needs info".

Data and API

- Bugs are work items with type = bug plus a bug detail extension (or equivalent). GET/POST /api/v1/bugs, POST /bugs/:id/verify.

States and edge cases

- Duplicate candidate suggestion, cannot reproduce, regression, reopened defect.

Acceptance criteria

- Closing a bug requires a verification record.
- Reopen increments reopen count.

7.5 QA test management

Purpose	Manage test cases, runs, results and traceability to requirements.
Primary users	QA engineers, developers, project leads.
Access rules	Project QA permissions; evidence retained per audit policy.

Screen layout and components

- Test case library: steps, expected results, priority, automation flag.
- Run page: environment/build, tester, per-case result (pass, fail, blocked, not run).
- Coverage panel linking cases to stories, bugs and releases.

User actions and behaviour

- Create/edit cases; execute runs; attach evidence; file a bug directly from a failed step; schedule regression suites; compare runs; mark obsolete; view requirement-to-test traceability.

Business rules

- Filing a bug from a failure prefills steps, environment and build and links the result.
- Flaky test marker excludes a case from pass-rate gates until fixed (still visible).
- Automated results may be imported through the API (CI) with a service token.

Data and API

- GET/POST /api/v1/test-cases, /test-runs; PATCH /test-runs/:id/results; POST /test-results/import.

States and edge cases

- No suite, flaky test, environment unavailable, blocked test.

Acceptance criteria

- Every requirement shows linked test cases and last result.

7.6 Release management and deployment overview

Purpose	Plan releases with readiness and traceability; show operational deployment activity.
Primary users	Release manager, PM, engineering, QA, DevOps, tech leads.
Access rules	Release management permission; environment and repository scope for deployments; secrets are never displayed.

Screen layout and components

- Release list: version, target date, status, owner, project. Detail: included work, unresolved bugs, linked PRs, test status, approvals, deployment history, readiness checklist, release-notes editor.
- Deployment overview: environment cards (dev, staging, production, custom), latest status, history with commit/PR, release version, actor, duration, failure reason; change calendar; deployment frequency summary.

User actions and behaviour

- Create release; assign items; generate draft notes; approve or hold; mark released; link deployment; roll back/withdraw with reason; compare planned vs actual date. Register deployment manually or by webhook; mark success/failed/rolled back; associate with release; filter by environment, repository, service, date.

Business rules

- Draft notes are generated from linked work items grouped by type; editor can modify.
- Deployment webhooks are authenticated with a per-environment secret/HMAC; duplicates ignored by delivery ID.
- DORA-style indicators (deployment frequency, change failure rate, time to restore) are computed from deployments and incidents.

Data and API

- GET/POST /api/v1/releases, POST /releases/:id/approve, /releases/:id/release; GET/POST /api/v1/deployments; POST /api/v1/deployments/webhook.
- Tables: releases, release_items, deployments, environments, workflow_runs.

States and edge cases

- Blocked by critical defect, missing approval, partial rollout, rollback, webhook delay, failed workflow, unknown commit.

Acceptance criteria

- A release with an open critical bug cannot be approved without recorded exception.
- A deployment arriving twice appears once.

7.7 Work log / time tracking

Purpose	Capture effort against work without rewarding excessive hours.
Primary users	Employees, team leads, PMs.
Access rules	Users manage own entries; managers/admins see scoped summaries by policy.

Screen layout and components

- Log form: item, date, duration, description, billable flag only if enabled.

- Worklog list by person, item, project, duration, date; summaries by user/project/period.

User actions and behaviour

- Add/edit/delete own entries within policy; managers correct only with audit trail; filter and export permitted data; compare estimate with actual.

Business rules

- Hours worked must never be used as a performance score input (Source requirement).
- Entries in a locked accounting period are read-only. Max duration per entry and per day is configurable to catch mistakes.

Data and API

- GET/POST /api/v1/worklogs; PATCH/DELETE /api/v1/worklogs/:id.

States and edge cases

- Invalid duration, locked period, duplicate entry, edited-entry audit.

Acceptance criteria

- A manager edit creates an audit entry and notifies the employee.

7.8 Knowledge base (documentation)

Purpose	Keep project and engineering knowledge next to delivery work.
Primary users	All project members; document owners and admins.
Access rules	Document-level access may be stricter than project access.

Screen layout and components

- Space/folder navigation, document list, search, editor with headings, tables, code blocks, attachments.
- Templates: project brief, architecture decision, runbook, meeting notes, onboarding guide, release notes.
- Metadata: owner, labels, project/team, version, last updated, access.

User actions and behaviour

- Create/edit/comment; link to work item, project or repository; publish/archive; restore previous version; manage access; template use; search authorized docs.

Business rules

- Each save creates a version; publish marks the current version visible.
- Concurrent editing: at minimum, conflict detection with merge prompt; real-time co-editing is optional.

Data and API

- GET/POST /api/v1/documents; GET /documents/:id/versions; POST /documents/:id/restore.

States and edge cases

- Draft, published, archived, edit conflict, restored version, inaccessible attachment.

Acceptance criteria

- A restricted document does not appear in search for unauthorized users.

7.9 Discussions, comments and mentions

Purpose	Contextual collaboration on work, projects, documents and retrospectives.
Primary users	Project and team members.
Access rules	Thread visibility follows the parent resource; private/internal comments have separate permission.

Screen layout and components

- Threaded comments, mentions, reactions, attachments, rich text, edit history; activity timeline.

User actions and behaviour

- Comment, reply, mention, react, edit/delete own content subject to policy; subscribe/unsubscribe; convert a discussion action into a task; report inappropriate content if policy supports.

Business rules

- Mentioning someone who lacks access does not reveal the resource; the author is warned and the mention is not delivered.
- Edits keep history; deletes are soft with tombstone "comment removed".
- Sanitize rich text on the server (allow-list HTML) to prevent stored XSS.

Data and API

- GET/POST /api/v1/work-items/:id/comments; PATCH/DELETE /api/v1/comments/:id; POST /comments/:id/reactions.

States and edge cases

- Deleted user, removed attachment, mention target lacks access, edit window expired.

Acceptance criteria

- Rich-text input containing script tags is stored and rendered safely.

8. GitHub Integration and Engineering Activity

GitHub is an **evidence and workflow integration**. DevTeam Hub supports personal GitHub authorization and organization-level GitHub App installation with explicit repository selection and visible permissions. GitHub data enriches work items and analytics but is never used as a standalone productivity score.

8.1 Integration architecture

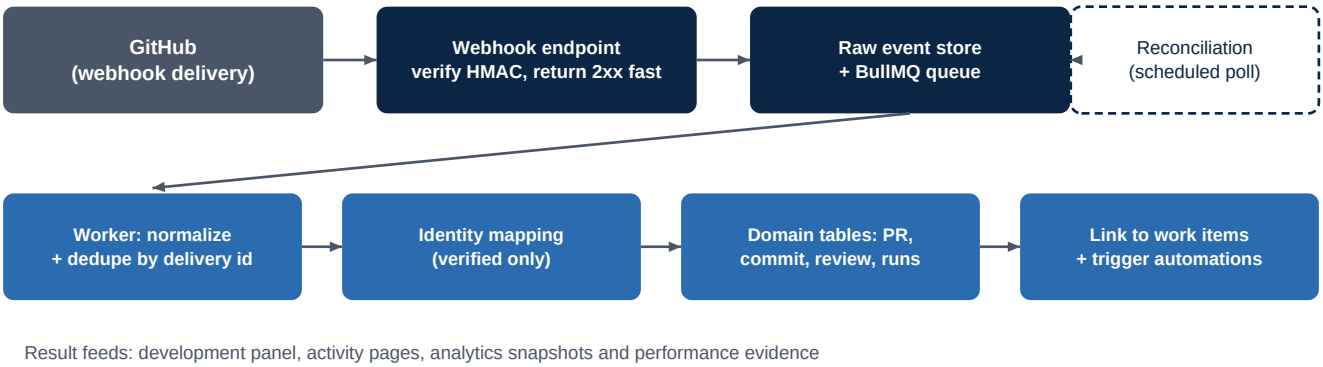


Figure 4 - GitHub ingestion pipeline. Webhooks give near-real-time updates; scheduled reconciliation repairs missed or failed deliveries.

Mechanism	Used for	Details
GitHub App installation (organization)	Repository access, webhooks, sync	Installed by an organization admin on the company GitHub organization with explicit repository selection. The server signs a short-lived JWT with the App private key to obtain installation access tokens (about 1 hour). Private key lives in the secrets manager/KMS.
OAuth / user authorization (personal)	Linking an individual GitHub identity to an employee	User authorizes with minimal scopes. Used to verify who owns a GitHub account, not to read private repositories beyond what the App grants. Tokens are encrypted; refresh tokens rotated where supported.
Webhooks	Near real-time events	Single endpoint secured with HMAC signature. Acknowledge quickly; process asynchronously.
Scheduled reconciliation	Missed events and backfill	Per repository poll using conditional requests and "since" cursors; compare counts; fix gaps.

Recommended GitHub App permissions (least privilege, read-first)

Permission	Level	Reason
Metadata	Read	Repository list and basic metadata (mandatory)
Contents	Read	Branches, commits (not required to clone code)
Pull requests	Read (write only if commenting back is approved)	PR status, reviews, linked items
Checks and Commit statuses	Read	CI status in development panel
Actions	Read	Workflow runs for deployment and failure insight
Deployments	Read	Deployment records and status
Issues	None unless GitHub Issues sync is approved	Avoid unnecessary scope

Subscribe to events: **installation, installation_repositories, repository, push, create, delete, pull_request, pull_request_review, pull_request_review_comment, check_run, check_suite, workflow_run, deployment, deployment_status, release**. The UI must clearly list what DevTeam Hub reads and what it can write (Source).

Webhook handling (mandatory steps)

```
// NestJS: enable raw body for this route so the signature is computed over the exact bytes
import { createHmac, timingSafeEqual } from 'crypto';

function isValidSignature(rawBody: Buffer, header: string | undefined, secret: string): boolean {
  const expected = 'sha256=' + createHmac('sha256', secret).update(rawBody).digest('hex');
  const a = Buffer.from(header ?? '');
  const b = Buffer.from(expected);
  return a.length === b.length && timingSafeEqual(a, b);      // constant-time comparison
}

// handler outline
// 1. verify signature  -> else 401, store nothing sensitive
// 2. read X-GitHub-Delivery, X-GitHub-Event
// 3. INSERT raw event (delivery_id UNIQUE) ON CONFLICT DO NOTHING   -> duplicate = already received
// 4. enqueue job { eventId }  and return 202 immediately
```

- **Deduplicate** by provider delivery ID (and by object version where relevant). Store both the provider timestamp and the ingestion timestamp.
- **Order is not guaranteed.** Worker logic must be idempotent and use the PR/commit state from the payload or a follow-up API call, never assume event order.
- **Reconciliation**: nightly (and on demand) compare GitHub state with stored state for active repositories and repair differences. Show "last sync" and "webhook health" to admins.
- **Rate limits**: use installation tokens, conditional requests (ETag), GraphQL for batched reads, and back off on secondary limits. Show "sync delayed" rather than failing user actions. Verify current limits in GitHub documentation during implementation.
- **Work item linking**: detect keys such as DTH-123 in branch names, PR titles/bodies and commit messages with a strict pattern, and accept only keys that exist in the organization. Manual link/unlink is always possible.
- **Bots**: Dependabot and other bot accounts are labelled and excluded from people-based metrics.

Identity mapping (employee to GitHub account)

- Never infer identity from display name or email text alone (Source).
- Verified methods: (1) the employee authorizes via OAuth and the returned GitHub user ID is stored; (2) an admin maps an account and the employee confirms; (3) verified commit email matched and confirmed by the employee.
- Multiple GitHub accounts may map to one employee only after verification of each.
- Unmatched events stay attributed to the GitHub login with status **unmapped** and are excluded from person-level metrics until mapped.
- Users and managers can report an incorrect mapping; corrections are audit-logged and trigger recomputation of affected snapshots.

Disconnect and retention

- Disconnecting or uninstalling stops future ingestion immediately and revokes stored tokens.
- Previously synchronized data is retained or deleted according to the organization retention policy (configurable: keep as history, anonymize, or delete). Linked evidence inside historical performance snapshots is preserved as snapshot data unless the policy requires deletion.

8.2 GitHub connections hub

Purpose	Show all GitHub identities and installations connected to the organization.
Primary users	Organization admins, integration managers, users connecting personal accounts.
Access rules	Only the connection owner/admin manages credentials. Secrets never shown after creation.

Screen layout and components

- Cards: personal account OAuth, organization GitHub App installation, status. Repository access summary, last sync, granted scopes, webhook health, reconnect/disconnect.
- Personal and organization connections are labelled differently.

User actions and behaviour

- Connect personal account; install/authorize App; select repositories; reauthorize; pause sync; disconnect; view sync logs; validate webhook; map repositories to projects/teams.

Business rules

- Organization connections require admin permission; personal connections are visible only to their owner and admins (without tokens).
- Disconnect asks for confirmation and explains the effect on stored data.

Data and API

- POST /api/v1/github/connections; GET /github/connections; POST /github/connections/:id/pause; GET /github/connections/:id/logs.
- Tables: github_accounts, github_connections, github_installations, webhook_deliveries.

States and edge cases

- Expired token, removed installation, repository access revoked, sync paused, webhook failing.

Acceptance criteria

- A revoked installation shows a clear "reconnect" state and stops sync.

8.3 Repository directory and repository detail

Purpose	Browse repositories available through connected accounts and see engineering activity for one repository.
Primary users	Integration managers, tech leads, developers, authorized project members.
Access rules	Only repositories granted to the connection and authorized to the user appear.

Screen layout and components

- Directory: name, owner/org, visibility, default branch, language, last activity, mapped project; filters by source, organization, visibility, team, mapped/unmapped; health indicator.
- Detail: metadata, mapped project/team, branches, recent commits, pull requests, releases, workflow runs, contribution activity by person (with source and time window), sync status.

User actions and behaviour

- Map/unmap to project or team; sync options; refresh metadata; open commit/PR on GitHub; filter activity; associate work items; inspect failed runs; report incorrect identity mapping.

Business rules

- Prevent duplicate or conflicting mappings unless explicitly supported (a repo may map to several projects only when enabled).
- Repository visibility inside DevTeam Hub follows GitHub grants plus DevTeam Hub permissions: both must allow.

Data and API

- GET /api/v1/github/repositories; POST /github/repositories/:id/mapping; GET /github/repositories/:id/activity.
- Tables: repositories, repository_mappings, commits, pull_requests, workflow_runs.

States and edge cases

- Repository deleted, access revoked, archived, mapping conflict, no activity, sync lag, identity unmatched.

Acceptance criteria

- Unmapping stops linking new events but keeps history.

8.4 Pull request detail and development panel

Purpose	Connect a pull request to delivery work and its review lifecycle.
Primary users	Developers, reviewers, QA, project leads.
Access rules	Only permitted repository and PR information; review content access follows the connection scope.

Screen layout and components

- PR title/number, repository, author, reviewers, state, draft flag, branch, checks, merge date.
- Linked work items, commits, review approvals/comments summary, CI status, deployment and release links. Embedded as a "Development" panel in the work item detail page.

User actions and behaviour

- Link/unlink work item; request review; open PR in GitHub; refresh status; flag PR with no linked work item when traceability policy requires.

Business rules

- Status comes from webhook with polling fallback; show a timestamp of last update.
- Force-pushes update the commit list and invalidate stale check results.

Data and API

- GET /api/v1/github/pull-requests/:id; POST /github/pull-requests/:id/links.

States and edge cases

- PR closed without merge, force-push, stale check status, missing identity mapping.

Acceptance criteria

- Merged PRs appear on linked work items within the target sync delay.

8.5 GitHub activity and contribution page

Purpose	Show meaningful engineering contributions with context rather than raw volume.
Primary users	Employees and managers within authorized scope.
Access rules	Individual private view by default where policy requires; manager scope is permission-controlled.

Screen layout and components

- Timeline of commits, PRs, reviews, releases and workflow events; filters by user, repository, project, type, date.
- Contribution cards: linked work item, PR outcome, review participation, release association.

User actions and behaviour

- Open source event; correct identity mapping; link missing work item; export scoped activity; report duplicate or misattributed events.

Business rules

- Do not show counts as rankings. Show context (what was delivered) instead of totals.
- Bot activity labelled and excluded from people metrics.

Data and API

- GET /api/v1/github/activity?user=&repo;=&type;=&from;=&to;=.

States and edge cases

- Delayed events, ambiguous identity, non-attributable contribution.

Acceptance criteria

- A manager outside the employee's scope cannot open that employee's activity.

Rule from the source
Never calculate employee performance from raw lines of code, raw commit count or PR count alone.

9. Analytics and Performance Insights

9.1 Metric catalog (define once, reuse everywhere)

Each metric needs one written definition, a data source, a calculation window, inclusion/exclusion rules and an owner. Store definitions in **metric_definitions** with version numbers; when a definition changes, show a note on charts and keep old snapshots unchanged.

Metric	Definition (recommended)	Source
Throughput	Work items moved to Done in the period (optionally weighted by size).	Work items
Cycle time	Time from first In Progress to Done for an item. Report median and 85th percentile, not just mean.	History
Lead time	Time from creation (or commitment) to Done.	History
WIP and aging	Items in progress now; age of each since entering In Progress.	Work items
Sprint predictability	Completed committed points / committed points at sprint start.	Sprint snapshots
Blocked time	Total time items spent flagged blocked.	History
Reopen rate	Items reopened after Done / items completed in period.	History
Escaped defects	Bugs found in production linked to work released in the period.	Bugs, releases
PR lead time	First commit (or PR open) to merge.	GitHub
Review turnaround	Review requested to first substantive review response.	GitHub
Deployment frequency	Successful production deployments per day/week.	Deployments
Change failure rate	Failed or rolled back production deployments / total.	Deployments
Time to restore	Time from failed deployment/incident to recovery.	Deployments, incidents
Capacity utilization	Committed estimate / available capacity (for planning, not for ranking).	Sprints, availability

9.2 Reports and analytics landing

Purpose	Directory of reports with access-aware scope and reusable filters.
Primary users	All users; managers and admins see more reports within scope.
Access rules	Report-level and row-level access. Export permission is separate from view permission.

Screen layout and components

- Categories: delivery, Agile, quality, engineering, release, workload, capacity, time tracking, performance.
- Saved reports, recent, shared dashboards, export history. Global filters: period, scope, department, team, project, repository.

User actions and behaviour

- Open, save filter, schedule report (if enabled), export CSV/PDF, build a custom report from approved dimensions and measures, drill down from a chart to authorized records.

Business rules

- Custom reports can only use approved dimensions and measures (no raw SQL).
- Drill-down re-applies the viewer's permission filter, so a chart total can be higher than the visible rows; show a "some data hidden" notice.

Data and API

- GET /api/v1/reports; GET /reports/:id; POST /reports; POST /reports/:id/export (async).

States and edge cases

- No data in period, delayed snapshot, insufficient sample size.

Acceptance criteria

- Export contents match what the user could view on screen.

9.3 Organization, department, team and project analytics

Purpose	Show delivery health, quality, engineering flow and workload at each level without exposing restricted detail.
Primary users	Org owner/admin and authorized leadership (org); department head (department); team members and leads (team); project members and leads (project).
Access rules	Each level requires explicit scope. Small cohorts are suppressed. Individual-level drill-down needs a separate permission.

Screen layout and components

- Organization: portfolio delivery, work health, quality, engineering flow, drill-down to departments/teams/projects.
- Department: team comparison, throughput and cycle-time trend, blocked/overdue, quality and release metrics, capacity, goals.
- Team: sprint predictability, throughput, cycle/lead time, aging, blocked time, WIP, reopened and escaped defects, review latency, workload by member (privacy controlled).
- Project: scope progress, milestone health, sprint outcomes, overdue/blocked, cycle time, defect trend, PR flow, release status, risk list with owner.

User actions and behaviour

- Change date range; compare periods; drill into items/PRs; annotate incidents, holidays and reorganizations; set team improvement goal; set project health with rationale; create corrective work; export aggregate views.

Business rules

- **Small-cohort suppression:** if a breakdown would describe fewer than N people (default N = 5, configurable), hide or aggregate it so individuals cannot be identified.
- Comparing teams is shown with context (team size, work type mix); avoid single-number rankings of teams.
- Metric trends use snapshots; a team moved between departments keeps historical membership.

Data and API

- GET /api/v1/analytics/organization|departments/:id|teams/:id|projects/:id with ?from=&to;=&compare;=.
- Tables: analytics_snapshots, metric_definitions, dashboards.

States and edge cases

- Partial integrations, metric definition change, low sample size, missing estimates, changed workflow, stale integration data.

Acceptance criteria

- A department head cannot access another department's numbers.
- Suppressed cohorts show an explanatory message.

9.4 Performance insight: principles

- **Informational, not automatic.** Scores and rankings are management aids, never an automatic basis for compensation, promotion or termination (Source).
- **Explainable.** Every score shows category weights, normalized values, evidence links, data completeness, sample size and model version.
- **Fair by design.** Role-specific models; adjust for complexity, part-period work, approved leave, external blockers and approved scope changes. No raw commits, lines of code or hours worked.
- **Disputable.** Employees can add context and challenge incorrect evidence; corrections are audited.

- **Private by default.** Individual rankings private; the organization admin enables team or company leaderboards deliberately, per level.
- **Reproducible.** Period snapshots store model version, weights and inputs. Recalculation creates a new version rather than overwriting.
- **Pilot first.** Launch only after a pilot validates fairness and employee trust (Source).

9.5 Scoring model (Source weights, Recommended mechanics)

Category	Default weight	Evidence (Source)	Example metrics (Recommended)
Task completion and delivery	25%	Accepted work completed, complexity-adjusted delivery, milestone contribution, cycle-time context	Complexity-weighted accepted items vs plan; milestone items delivered; cycle time vs team baseline (capped influence)
Work quality	20%	QA acceptance, escaped defects, reopened work, rework, review findings	Reopen rate; escaped defects attributed after root-cause review; rework ratio; first-time QA pass rate
Code contribution	15%	Meaningful merged PRs, task-linked contributions, maintainability, technical outcomes	Merged PRs linked to work weighted by size bucket and complexity (capped); share of PRs with linked item; change-request rounds
Review and collaboration	15%	Review quality, helpful feedback, turnaround, pairing, mentoring	Substantive reviews completed; response time vs SLA; peer/manager-confirmed mentoring
Timeliness and reliability	10%	Commitment reliability adjusted for approved scope changes, blockers, dependencies	Committed items completed / committed (excluding approved changes and external blockers)
Problem-solving and initiative	10%	Critical issue resolution, automation, technical debt, improvements	Items tagged critical/tech-debt/automation completed; confirmed improvement outcomes
Team contribution	5%	Documentation, knowledge sharing, retrospectives, support	Docs published/updated; retrospective actions completed; support contributions

Weights are configurable per organization and role and must be calibrated with real data and reviewed for unintended bias before launch. Illustrative role variations (to be validated):

Category	Developer	QA	DevOps
Delivery	25	20	20
Quality	20	30 (defect detection effectiveness, verification accuracy)	15 (change failure rate)
Code / automation contribution	15	5 (test automation)	10 (infrastructure as code)
Review and collaboration	15	15	10
Timeliness and reliability	10	10	20 (restore time, deployment reliability)
Initiative	10	10	15
Team contribution	5	10	10

Calculation steps

- **1. Collect evidence** for the period from work items, QA, GitHub, reviews, goals and manager input; exclude approved leave days, bot activity and unmapped events.
- **2. Check eligibility**: employee active for at least a configurable share of working days (default 60%), minimum evidence count per category, and data completeness at least 70% overall. Otherwise show **Not ranked: insufficient evidence** with the reason.
- **3. Normalize each metric** to 0-100 using role-specific target bands (floor, target, stretch). Score = 0 at floor, 70 at target, 100 at stretch, linear between. Using targets (criterion-referenced) rather than pure percentile ranking avoids forcing losers in every cohort.

- **4. Apply small-sample shrinkage**: adjusted = (n x value + k x cohort_median) / (n + k), with k default 5. This stops three lucky items from producing an extreme score.
- **5. Category score** = weighted average of its metric scores; categories with too little evidence are marked incomplete.
- **6. Overall score** = sum of (category weight x category score). If a category is incomplete, redistribute its weight only if overall completeness stays above the threshold, and show that redistribution.
- **7. Assign band**: Exceptional 90-100, Strong 75-89, Solid 60-74, Developing 40-59, Below 40 (labels configurable). Show confidence (high/medium/low) from sample size and completeness.
- **8. Store snapshot** with model version, weights, inputs, evidence IDs and calculation timestamp.

Worked example (illustrative developer): category scores Delivery 82, Quality 76, Code 70, Review 88, Timeliness 80, Initiative 65, Team 90.

Category	Weight	Score	Contribution
Delivery	25%	82	20.5
Quality	20%	76	15.2
Code contribution	15%	70	10.5
Review and collaboration	15%	88	13.2
Timeliness	10%	80	8.0
Initiative	10%	65	6.5
Team contribution	5%	90	4.5
Overall	**100%**		**78.4 - Strong**

Anti-gaming and fairness controls

- Cap the influence of any count-based signal (PRs, reviews) and weight by size/complexity; ignore trivial or auto-generated changes (formatting-only, dependency bumps) for contribution scoring.
- Reviews count only when substantive (approved with comments, requested changes, or resolved discussion), not rubber-stamp approvals within seconds.
- No hours-worked or after-hours activity input. No keystroke or screen monitoring.
- Check score distribution by team, role, tenure, location and part-time status; investigate systematic differences before launch and after each model change.
- Complexity comes from estimates or size buckets set before work starts (not after) to avoid inflation.
- Attribute defects through a root-cause review step, not automatically to the last committer.

9.6 Individual performance overview and score detail

Purpose	Give an employee transparent insight into outcomes, growth and evidence; explain how a score was produced.
Primary users	Employee (own); managers for authorized reports; HR/admin by policy.
Access rules	Self view always. Manager access by reporting/team scope. Score details are more restricted than aggregate analytics.

Screen layout and components

- Score summary and band (if enabled), period selector, role model and model version.
- Category cards: weight, normalized score, contribution, completeness; each metric shows definition, source, period, weight, normalized value, sample size and linked evidence.
- Goals, completed work, quality outcomes, collaboration evidence, trend, excluded data and adjustment history.

User actions and behaviour

- Open evidence; read metric definition; add context; dispute evidence; set personal goals; acknowledge feedback; view past periods; managers add feedback or propose a correction with reason; download explanation where permitted.

Business rules

- Dispute states: Open, Under review, Accepted (recalculated), Rejected with reason, Escalated. Target SLA for review (for example 5 working days).
- Score adjustments require reason, author, and approval by a second authorized person; they never delete original evidence and are visible to the employee.
- Every manager view of an employee's detail is audit-logged.

Data and API

- GET /api/v1/performance/me; GET /performance/users/:id; POST /performance/feedback; POST /performance/disputes; POST /performance/adjustments (+ approve/reject).
- Tables: performance_periods, performance_weights, performance_scores, performance_score_details, score_adjustments, performance_feedback, performance_goals.

States and edge cases

- Not enough evidence, new employee, role changed mid-period (prorate by role), data source disconnected, metric unavailable, event misattributed, model updated after snapshot, disputed score.

Acceptance criteria

- Each score can be traced to its evidence in two clicks.
- A manager outside scope receives not-found.

9.7 Leaderboard (organization / department / team)

Purpose	Configurable rankings for a selected cohort and period, only where the organization explicitly enables them.
Primary users	Only users authorized by the organization visibility policy.
Access rules	Recommended default: individual rankings private; org admin enables team/company leaderboard; visibility can differ by level.

Screen layout and components

- Rank or band, display name/avatar, role/team, score band, trend, category highlights.
- Filters: department, team, project, role, month/quarter/year. Info: model version, eligible population count, evidence completeness, methodology link.
- Option to show team-level aggregate ranking instead of individuals.

User actions and behaviour

- Change cohort and period; open individual detail if permitted; export only when authorized; report a score issue; view methodology.

Business rules

- Do not rank users with insufficient evidence; show reason instead.
- Tie handling: equal scores share a rank (1, 1, 3). For small cohorts (under N) hide ranking and show bands only.
- Recommended: public leaderboards show bands and category highlights, not exact decimals, to reduce unhealthy competition.
- Rankings use period snapshots; late data corrections create a new snapshot version with a visible note.

Data and API

- GET /api/v1/performance/leaderboard?scope=.=; GET/PATCH /performance/visibility-settings (admin).
- Tables: performance_rankings, ranking_visibility_settings, performance_badges, employee_badges.

States and edge cases

- Ties, small cohorts, insufficient sample, leave or part period, role change, missing data.

Acceptance criteria

- With leaderboards disabled the page and endpoint return not-found for non-admins.
- Employees with insufficient evidence are never ranked.

9.8 Goals and feedback

Purpose	Support development conversations and measurable goals alongside quantitative signals.
Primary users	Employees, managers, HR roles as configured.
Access rules	Employees see their own; managers see scoped reports; HR follows policy. Confidential feedback has separate visibility.

Screen layout and components

- Goal list: title, outcome, target, due date, status, progress. Feedback entries: strengths, improvement areas, evidence, follow-up. Check-in history and review cycles.

User actions and behaviour

- Create/update goals; comment; acknowledge feedback; manager records check-ins; link goals to projects or measurable outcomes; close or revise with history.

Business rules

- Goals and feedback are not auto-generated from scores. A review period links goals, feedback and the score snapshot for context.
- When a reporting relationship changes, old manager access to history ends per policy and is audit-logged.

Data and API

- GET/POST /api/v1/performance/goals; POST /performance/feedback; POST /performance/feedback/:id/acknowledge.

States and edge cases

- Goal overdue, relationship change, confidential feedback.

Acceptance criteria

- Employees can acknowledge feedback; acknowledgement is stored with timestamp.

10. Administration, Automation and Integrations

10.1 Organization settings

Purpose	Configure organization identity, policies, structure and lifecycle.
Primary users	Organization owner and admin.
Access rules	Owner/admin only; sensitive changes require confirmation (and re-authentication) and audit.

Screen layout and components

- Company name, logo, verified domain; timezone and locale defaults; departments; working calendar.
- Membership policy, invitation rules, default roles, project visibility, data retention.
- Feature controls: performance leaderboard, analytics visibility, integrations.

User actions and behaviour

- Update settings; manage domain and SSO; set default policies; export organization data; transfer ownership; suspend or archive organization with safeguards.

Business rules

- There must always be at least one owner; the last owner cannot be removed or demoted.
- Ownership transfer requires the new owner to accept and the current owner to re-authenticate.
- Domain verification via DNS TXT record before domain-based join or SSO enforcement.

Data and API

- GET/PATCH /api/v1/settings; POST /settings/domains/verify; POST /organizations/:id/transfer-ownership.

States and edge cases

- Domain verification pending, policy conflict, active user dependencies.

Acceptance criteria

- Changing a policy writes an audit event with before/after values.

10.2 User and access administration

Purpose	Manage the employee lifecycle and permissions.
Primary users	Organization admin; HR/admin role if configured.
Access rules	Admin permissions; high-risk changes may require re-authentication or second approval.

Screen layout and components

- User table: name, email, status, role, department, teams, last sign-in, invitation state.
- User detail: memberships, roles, project grants, GitHub identity links, access history. Bulk invite, deactivate, reactivate, role assignment.

User actions and behaviour

- Invite; change role; move department/team; suspend; revoke sessions; review permission summary before saving; export directory if allowed.

Business rules

- Deactivation (offboarding) revokes sessions and API tokens, unassigns or reassigns open work with a prompt, keeps history and attribution.
- An effective-permissions preview must show what the user will be able to do after the change.
- Duplicate emails are rejected per organization.

Data and API

- GET/POST /api/v1/users; PATCH /users/:id; POST /users/:id/deactivate; POST /users/:id/revoke-sessions.

States and edge cases

- Duplicate email, pending invite, last owner removal, user owns integration (reassign first).

Acceptance criteria

- Deactivated users cannot sign in or use tokens immediately.

10.3 Roles and permissions

Purpose	Define reusable role presets and custom access policies.
Primary users	Organization admin and security admin.
Access rules	Admin/security permission; some actions owner-only.

Screen layout and components

- Role list; permission matrix by resource and action; scope selector (organization, department, team, project, repository, personal); effective-access preview for a selected user.

User actions and behaviour

- Create/clone role; edit permissions; assign role; compare role changes; warn on broad access; audit all changes.

Business rules

- Permission keys are defined in code (resource:action, for example work_item:transition) and stored as data; custom roles can only combine existing keys.
- System roles (Owner) cannot be edited. Changing a role takes effect immediately by invalidating permission caches.
- Least-privilege linter: warn when a role grants organization-wide write or any export permission.

Data and API

- GET/POST /api/v1/roles; PATCH /roles/:id; POST /role-assignments; GET /users/:id/effective-permissions.

States and edge cases

- Conflicting rules (deny wins), inherited deny, orphaned role assignment.

Acceptance criteria

- Effective-permission preview matches actual API behaviour (tested).

10.4 Workflow and field configuration

Purpose	Let admins tailor work types, statuses, transitions and required fields.
Primary users	Organization and project admins.
Access rules	Admin access; project overrides cannot violate organization-wide mandatory controls.

Screen layout and components

- Workflow designer: statuses, transitions, validators, conditions. Work type schemes, field definitions, required/optional, screen layouts. Preview and migration impact.

User actions and behaviour

- Create workflow version; test transitions; publish; migrate active items; configure fields and type mappings; roll back where feasible.

Business rules

- Workflows are versioned. Publishing a new version requires mapping each removed status to a surviving status for active items.
- Validators available: required field, assignee set, linked PR, PR approved, sub-tasks done, approval granted, user role. Post-functions: set field, notify, assign, trigger automation.

Data and API

- GET/POST /api/v1/workflows; POST /workflows/:id/publish; GET/POST /api/v1/fields; /work-types.

States and edge cases

- Invalid transition graph (unreachable or dead-end status), active items in removed status, incompatible field migration.

Acceptance criteria

- Publishing an invalid graph is blocked with a readable error.

10.5 Automation rules

Purpose	Automate repetitive work with triggers, conditions and actions.
Primary users	Project admins, organization admins, authorized team leads.
Access rules	Rule creation scoped; actions run under a controlled service identity with explicit permissions.

Screen layout and components

- Rule library: name, scope, owner, enabled, last run. Rule builder: trigger, conditions, actions, schedule, execution limits. Execution history: input, result, error, retry.

User actions and behaviour

- Create, test (simulation), enable/disable; inspect logs. Actions: assign, transition, notify, set field, create linked item, call approved integration.

Business rules

- Triggers: work item created/updated/transitioned, PR opened/merged, deployment finished, schedule, webhook.
- Loop prevention: a rule execution carries an origin chain ID; a rule cannot fire from changes made by itself within the chain; max depth and per-rule rate limits.
- Outbound calls only to allow-listed integrations; no arbitrary URL calls (SSRF protection).

Data and API

- GET/POST /api/v1/automation-rules; POST /automation-rules/:id/simulate; GET /automation-executions.

States and edge cases

- Loop detected, rate limit, permission denied, retry exhausted, disabled rule.

Acceptance criteria

- A rule that transitions an item on PR merge works and is logged with input and result.

10.6 Notifications and inbox

Purpose	Collect actionable updates without overwhelming users.
Primary users	All users.
Access rules	Only the recipient sees private notifications; target access is checked on open.

Screen layout and components

- Inbox tabs: unread, mentions, assigned, watched, approvals, system. Preferences: in-app, email, push, digest frequency. Item shows actor, event, target, time, action.

User actions and behaviour

- Open target; mark read/unread; mute thread; bulk mark read; unsubscribe where permitted; configure preferences.

Business rules

- Critical security notices cannot be muted when policy requires.
- Deduplicate repeated events (for example many comments in 2 minutes) into one notification.

Data and API

- GET /api/v1/notifications; PATCH /notifications/:id; GET/PUT /api/v1/notification-preferences.

States and edge cases

- Target deleted, access revoked, delivery failed, duplicate event.

Acceptance criteria

- Opening a notification for a resource the user lost access to shows a safe message, not the content.

10.7 Integrations marketplace and API access

Purpose	Manage external services and developer access.
Primary users	Organization admin, integration manager, developer admin.
Access rules	Admin/integration permission; secrets visible only at creation.

Screen layout and components

- Catalog of integrations; connected apps with status and scopes; API tokens, webhooks, OAuth apps, event subscriptions, docs; rate limits, usage, last delivery.

User actions and behaviour

- Connect/disconnect; rotate token; create/revoke API key; configure webhook endpoint and events; send test delivery; inspect retries; review scopes before granting.

Business rules

- API keys are scoped (read-only, project-limited), hashed at rest, shown once, expire by default, and can be revoked immediately.
- Outgoing webhooks are signed (HMAC) with a per-endpoint secret and retried with backoff; endpoints must be HTTPS and are validated against private-address ranges (SSRF).

Data and API

- GET/POST /api/v1/api-keys; POST /api-keys/:id/rotate; GET/POST /api/v1/webhooks; GET /webhooks/:id/deliveries.

States and edge cases

- Invalid credentials, expired token, unreachable endpoint, rate limit, revoked app.

Acceptance criteria

- A revoked key fails on the next request.

10.8 Audit log and security center

Purpose	Traceability for sensitive actions and visibility of security posture.
Primary users	Organization owner, admin, security admin.
Access rules	Security admin access; export may need elevated approval.

Screen layout and components

- Audit timeline: actor, action, resource, time, source (IP/device), outcome; filters by member, action, date, project, integration.

- Security panel: MFA/SSO coverage, active sessions, token status, policy exceptions.

User actions and behaviour

- Inspect event; export; revoke session or token; investigate access change; set retention; alert on selected events.

Business rules

- Audit events are append-only (see Section 12.10), include before/after for changes and never contain secrets or full personal data beyond identifiers.
- Reading the audit log is itself audited.

Data and API

- GET /api/v1/audit-logs; POST /audit-logs/export; GET /api/v1/security/overview; DELETE /sessions/:id.

States and edge cases

- Retention boundary, incomplete external event, export too large (async).

Acceptance criteria

- Role changes, exports, integration changes, score adjustments and failed logins all appear in the log.

11. Mobile Application and Shared Interaction Rules

The mobile app complements the web workspace. It optimizes quick updates, communication, notifications and approvals. Complex administration, workflow design, advanced reporting and bulk data management remain **web-first**.

Screen	Contents	Key behaviours and rules
11.1 Sign-in and organization switcher	Email/password or SSO, MFA, biometric unlock after secure initial sign-in, organization list with role and recent access, connection status	Same auth and tenant rules as web. Biometric only unlocks a locally stored refresh token. Re-authentication for sensitive settings. Sign-out revokes the device session. Offline indicator.
11.2 Mobile home	Cards: My Work, overdue, mentions, upcoming due dates, recently viewed; sprint progress and release alerts where authorized; bottom nav Home / My Work / Create / Inbox / Profile	Pull to refresh; quick-create; limited card customization. Cached summary when offline with last-synced time.
11.3 My Work and task detail	List with status/priority/due, filters and search; detail with summary, description, status, assignee, due date, comments, attachments, linked PR summary, compact history, dependency warning	Update status, comment, self-assign if allowed, checklist update, attach photo/file, log work, open full web view for complex edits, deep link to PR/repo. Same item permissions as web. Offline edits are queued and clearly marked until confirmed.
11.4 Create	Type, project, summary, description, priority, assignee, due date, optional photo/file, device voice dictation	Validate required fields; preserve draft if the user leaves; duplicate-submit prevention through idempotency key; network retry; advanced fields open on web.
11.5 Inbox and notifications	Grouped notifications with unread state; quick acknowledge, approve/reject, open item; preference shortcut	Push deep links route to the correct organization and resource (switch tenant first if needed). Permission re-checked at action time. Expired approvals and revoked access handled gracefully.
11.6 Profile and preferences	Photo, display name, title, department/team, timezone, notification preferences, theme, language, security and sessions, performance self-view shortcut (if enabled)	Edit permitted fields only; organization-managed fields are read-only; request profile correction; manage sessions; sign out.

11.7 Mobile-specific requirements

- Secure token storage (Keychain/Keystore). No sensitive performance or company data in plain local storage; cache only the minimum for recently viewed work; wipe on logout and organization switch.
- Distinguish queued changes from server-confirmed ones. Offline mode shows last-synced time and disables unsafe actions.
- Push notifications with tenant-aware deep links. Do not put confidential text in the push payload.
- Optional hardening: certificate pinning with a rotation plan, jailbreak/root detection warning for sensitive roles, screenshot blocking on performance screens (configurable), auto-lock timeout.
- Accessible touch targets (at least 44 x 44 pt), readable typography, dynamic text size, clear loading/error feedback.
- Release management: staged rollout, forced-update mechanism for security fixes, crash reporting without personal data.

11.8 Shared interaction, validation and notification rules

State	Expected behaviour
Loading	Skeletons or progress indicators; preserve context; avoid layout jumps.
Empty	Explain why there is no data and offer the next valid action (create, connect, clear filters).
No results	Keep filters visible; offer clear filters; distinguish no results from no permission.
Error	Safe, actionable message and retry; never expose stack traces or secrets.
Permission denied	Explain that access is restricted without revealing protected resource details.
Offline	Show last-synced timestamp; disable unsafe actions or queue supported edits explicitly.
Conflict	Show server version and local changes; allow refresh or merge where supported.
Success	Confirm the action and link to the created/updated resource.

- Validate on client and server; server validation is authoritative.
- Consistent date/time handling with clear organization and user timezone.
- Prevent duplicate submissions for create and destructive actions (idempotency keys, disabled buttons).
- Require confirmation for delete, archive, disconnect, role escalation and bulk operations.
- Optimistic updates only where rollback and conflict handling are reliable.
- Keyboard navigation, screen-reader labels, visible focus, sufficient contrast, non-colour status cues.
- Keep filters in URL state so views can be shared without leaking private data.

Notification event	Recipients	Default channel
Work item assigned / reassigned	New assignee; optionally watchers	In-app; email/push configurable
Mention or comment reply	Mentioned user / thread participants	In-app + push; email preference
Due date approaching / overdue	Assignee; optionally lead	In-app digest or configured reminder
PR review requested	Requested reviewer	In-app + push
PR merged / linked deployment failed	Watchers or project members by subscription	In-app
Sprint started / completed	Team members	In-app
Role / access changed	Affected user and admin audit channel	In-app / email
Security event	User, admin, security recipients	Immediate, policy-controlled alert

12. Security Design and Engineering Controls

DevTeam Hub stores company work data, source-control metadata, integration secrets and sensitive employee performance information. Security is therefore a **core feature**, not a final-phase task. This section lists the controls the team must implement and verify. Items are Recommended engineering controls consistent with the source requirements (least privilege, tenant isolation, audit, encryption, MFA/SSO, secret management).

12.1 Security objectives and threat model

Asset	Main threats	Primary mitigations
Tenant data (work items, docs, files)	Cross-tenant leak; broken object-level authorization (IDOR); insider over-access	Org-scoped queries + PostgreSQL RLS; central authorization; per-object ACLs; tenant-isolation test suite; audit
Accounts and sessions	Credential stuffing, phishing, session theft, token replay	Argon2id, MFA/SSO, rate limits, short-lived tokens with rotation and reuse detection, device session revocation
GitHub credentials and installation tokens	Private key or token theft; excessive scopes	Secrets manager + KMS, least-privilege App permissions, short-lived installation tokens, no secrets in logs, rotation
Performance data and rankings	Unauthorized viewing, misuse, bias, manipulation	Private by default, scope + policy checks, access auditing, score adjustments need second approval, snapshots immutable
Webhook and public endpoints	Forged events, replay, DoS	HMAC signature verification, delivery ID dedupe, rate limiting, fast ack + queue, WAF
Uploaded files	Malware, script execution, data exposure	Quarantine + AV scan, type allow-list, separate domain, signed URLs, authz on download
Automation and outbound integrations	SSRF, privilege escalation, loops	Allow-listed destinations, service identity with explicit permissions, rate limits, loop prevention
Infrastructure and supply chain	Misconfiguration, vulnerable dependencies, leaked CI secrets	IaC review, private networking, dependency/secret/container scanning, signed builds, least-privilege CI roles

12.2 Authentication

- **Password storage:** Argon2id with unique salt. OWASP minimum is about 19 MiB memory, 2 iterations, 1 degree of parallelism; recommended to tune upward (for example 64 MiB, 3 iterations) so a hash takes roughly 250-500 ms on production hardware. Support automatic rehash on login when parameters change. Add an optional server-side pepper held in the secrets manager.
- **Password policy:** minimum length 12 (passphrases encouraged), no composition gimmicks, check against breached-password list, no forced periodic rotation unless policy demands, block reuse of recent passwords.
- **MFA:** TOTP and WebAuthn/passkeys; hashed single-use recovery codes. Organization policy can require MFA for all or for privileged roles. Re-authentication (step-up) for sensitive actions: role changes, ownership transfer, exports, integration secrets, score adjustments.
- **SSO:** OIDC (Google, Microsoft Entra) and SAML where required. Validate issuer, audience, signature, nonce/state and expiry. Just-in-time provisioning only for verified domains. SSO-enforced organizations disable passwords. Optional SCIM later for automated deprovisioning.
- **Account protection:** rate limits and progressive delays per IP and per account, temporary lock after repeated failures with notification, generic error messages and constant-ish response time, CAPTCHA only after repeated failures.
- **Password reset and invitations:** random 256-bit tokens, stored hashed, single use, short expiry (reset 30-60 minutes; invitation days), invalidated on use or password change; reset never reveals account existence.
- **Email verification** before an address can receive sensitive notifications or be used for domain-based joining.

Session and token design

Item	Design
Access token	Short-lived (about 10-15 minutes) signed JWT (asymmetric keys with key ID and rotation) containing user ID, organization ID, session ID. Permissions are NOT embedded; they are resolved server-side so role changes apply immediately.
Refresh token	Opaque random token, stored hashed in the sessions table, rotated on every use. **Reuse detection:** if an old token is presented, revoke the whole token family and force re-login.
Web storage	Refresh token in **HttpOnly; Secure; SameSite=Lax (or Strict)** cookie scoped to the API domain; access token kept in memory only (never localStorage). CSRF protection for cookie-authenticated state-changing requests (CSRF token or custom header with SameSite).
Mobile storage	Keychain / Keystore only. Biometric unlock gates local access to the refresh token.
Session controls	Absolute lifetime (for example 30 days) and idle timeout (configurable by policy). Users and admins can list and revoke sessions; revocation is effective on next refresh and, for high-risk revocations, immediately via a deny-list keyed by session ID.
Organization context	Active organization ID is part of the session context and is verified against membership on every request. Switching organization issues a new token.

12.3 Authorization enforcement

- **Deny by default.** Every controller route is protected by global guards; public routes are explicitly marked and reviewed.
- **Object-level checks (BOLA/IDOR).** Never trust an ID from the client. Load the resource within the tenant scope and call **authorize(user, action, resource)** before returning or mutating it. List endpoints add the same restrictions as query filters.
- **Field-level control.** Use explicit response DTOs/serializers. Never return ORM entities directly. Restricted fields (private comments, performance details, token metadata) are excluded unless permitted.
- **Mass-assignment protection.** Accept only whitelisted fields in input DTOs (ignore or reject unknown properties). Never bind request bodies directly to ORM models; fields such as organization_id, role, created_by are set server-side.
- **Same response for not-found and forbidden** for resources whose existence is sensitive (private projects, other tenants) to prevent enumeration.
- **Elevated actions** (see 12.2 step-up) re-verify the user and are audit-logged.
- **Service identities** (automation, workers) have their own limited permission sets and never inherit a human user's full access.

```
@Injectable()
export class PermissionGuard implements CanActivate {
  constructor(private authz: AuthorizationService, private reflector: Reflector) {}

  async canActivate(ctx: ExecutionContext): Promise<boolean> {
    const req = ctx.switchToHttp().getRequest();
    const required = this.reflector.get<PermissionSpec>('permission', ctx.getHandler());
    if (!required) throw new InternalServerErrorException('Route missing @Permission');   // fail closed
    const resource = await this.authz.resolveResource(req, required);   // loads inside tenant scope
    const ok = await this.authz.can(req.user, required.action, resource);
    if (!ok) throw new NotFoundException();                // do not reveal existence
    return true;
  }
}

@Patch(':id')
@Permission({ action: 'work_item:update', resource: 'work_item', param: 'id' })
update(@Param('id') id: string, @Body() dto: UpdateWorkItemDto) { ... }
```

12.4 Tenant isolation checklist

- Every tenant table has organization_id with RLS enabled and forced (Section 4.6). The app DB role is not a superuser and not the table owner.
- Background jobs carry organizationId in the payload and set the tenant context before touching the database.

- Redis keys, search documents, object-storage paths and export files are all prefixed or filtered by organization.
- WebSocket subscriptions are authorized per room and rooms include the organization ID.
- Log lines and traces include organization ID for investigation; metrics are not labelled with high-cardinality tenant data unless needed.
- CI includes automated cross-tenant tests (read, write, delete, search, export, WebSocket, file download) that must pass for every release.

12.5 Data protection and cryptography

Control	Requirement
In transit	TLS 1.2+ (prefer 1.3) everywhere including internal service and database connections; HSTS; modern cipher suites; automated certificate renewal.
At rest	Encrypted database, backups, object storage and Redis persistence using KMS-managed keys. Separate keys per environment.
Field-level encryption	Encrypt integration secrets, OAuth/refresh tokens and webhook secrets with envelope encryption (data key encrypted by KMS key). Never return them in API responses; show only masked metadata.
Hashing	API keys and invitation/reset/refresh tokens are stored as hashes (SHA-256 of high-entropy random value is acceptable for tokens; Argon2id for passwords).
Key management	KMS with rotation, restricted key policies, separate roles for key admins and key users; document key-compromise procedure.
Backups	Encrypted, access-restricted, tested restores, retention aligned with privacy policy (deleted data ages out of backups on schedule).
Data minimisation	Collect only needed GitHub fields. Do not store source code contents; store metadata and diffs statistics only unless a feature requires more.

Data classification (recommended): *Restricted* = secrets, tokens, performance scores and feedback, audit logs; *Confidential* = work items, documents, employee profile fields, repository metadata; *Internal* = aggregate analytics without individual detail; *Public* = none by default. Controls (access, logging, retention, export) tighten with each class.

12.6 Secrets management

- No secrets in source control, container images, front-end bundles or logs. CI runs secret scanning and blocks merges on findings.
- Runtime secrets come from a managed secrets service (for example AWS Secrets Manager) injected at start; access through task/pod identity with least privilege.
- Rotate: database credentials, JWT signing keys (with overlap using key IDs), webhook secrets, API keys. Define rotation intervals and an emergency rotation runbook.
- Environment variable list is documented without values. Local development uses fake secrets only.
- GitHub App private key: stored only in the secrets manager (or used through KMS signing), never on developer machines.

12.7 Application and API security

Risk	Control
Injection (SQL/NoSQL/command)	Parameterized queries via ORM; no string-built SQL; if raw SQL is needed use bound parameters; validate sort/filter fields against allow-lists.
XSS	Framework output encoding; sanitize rich text server-side with an allow-list (and again on render); strict Content-Security-Policy without unsafe-inline where possible; avoid dangerouslySetInnerHTML or isolate it.
CSRF	SameSite cookies + CSRF token/custom header on state-changing cookie-authenticated requests; bearer-token API calls are not CSRF-prone.
CORS	Explicit allow-list of web origins; no wildcard with credentials.
Security headers	Content-Security-Policy, Strict-Transport-Security, X-Content-Type-Options: nosniff, Referrer-Policy, Permissions-Policy, frame-ancestors none (or allow-list), cache-control no-store on sensitive responses.
SSRF	Outgoing requests (webhooks, automation, link previews) go through a hardened client: allow-list or block private/link-local/metadata IP ranges, resolve DNS and re-check, no redirects to internal hosts, timeouts and size limits.
File upload	Size and type allow-list, content sniffing, AV scan, random object keys, separate download domain, attachment disposition, no server-side execution or image-processing of untrusted files without sandboxing.
Deserialization and input size	Strict schema validation (Zod/class-validator); request body size limits; reject unknown fields; limit pagination page size (for example max 100).
Business-logic abuse	Idempotency keys; transition validators; per-user and per-tenant quotas; export and bulk operation limits; anomaly alerts on mass downloads.
Error handling	Uniform problem-details responses; log details server-side with correlation ID; never leak stack traces, SQL or internal IDs.
Dependency risk	Lock files, automated updates, vulnerability scanning on every PR, review of new packages, pin GitHub Actions by commit hash.

Rate limiting (starting values, tune with data)

Endpoint group	Limit	Key
Login / MFA verify	5 per minute, 20 per hour	IP and account
Password reset / invite resend	3 per hour	Account and IP
General authenticated API	600 per minute	User (and per tenant ceiling)
Search	60 per minute	User
Exports / report generation	10 per hour, 2 concurrent	User and tenant
Public API keys	Configurable per key; headers return remaining quota	API key
Inbound webhooks	High ceiling but verify signature before any work; drop invalid early	IP / installation

12.8 Integration and webhook security

- Inbound webhooks: verify HMAC over raw body with constant-time compare, reject missing/invalid signatures, dedupe by delivery ID, apply payload size limit, respond quickly and process asynchronously.
- Outbound webhooks: HMAC-sign, include timestamp and delivery ID so receivers can reject replays, retry with backoff, HTTPS only.
- GitHub: least-privilege App permissions, explicit repository selection, token encryption, expiry handling, identity verification before attribution (Section 8.1).
- Third-party integrations are added through an allow-listed catalog with reviewed scopes; unused connections are disabled automatically after inactivity (configurable).

12.9 Protecting performance and personal data

- Performance endpoints require explicit permission and scope checks; they are excluded from general search and from default exports.
- Every manager/HR view of an individual's performance detail, every export, and every adjustment is audit-logged and visible to security admins; employees can see who viewed their detail if policy allows.
- Score adjustments need reason plus a second approver; original evidence and snapshots are immutable.
- Small-cohort suppression and band-only public display reduce re-identification and unhealthy comparison.
- Retention: performance snapshots kept per HR/legal policy; deletion/anonymization workflow when an employee leaves or requests erasure where legally applicable.
- Avoid sensitive attributes (age, gender, ethnicity, health, location of residence) in scoring inputs. Run the fairness checks in Section 9.5 before launch.
- Never use hours worked, keystrokes, screen capture, message content or off-work activity.

12.10 Audit logging

- **Events to log (minimum):** sign-in success/failure/lockout, MFA changes, password reset, session/token revocation, role and permission changes, membership changes, project visibility changes, integration connect/disconnect, API key and webhook changes, exports, retention changes, performance views/adjustments/disputes, ownership transfer, audit log access.
- **Record fields:** event ID, organization ID, actor (user or service), action, resource type and ID, before/after summary (no secrets), outcome, IP, user agent/device ID, request ID, timestamp (UTC).
- **Append-only** storage: application has INSERT and SELECT only on audit_logs (no UPDATE/DELETE). For tamper evidence add a hash chain (each record stores hash of previous record + content) and ship copies to write-once storage (for example S3 Object Lock) on a schedule.
- **Retention:** configurable (for example 1-7 years by policy), with legal-hold support. Exports require elevated approval and are themselves logged.
- **Separate** audit events (security/compliance) from product activity history (work item history). Both are append-oriented.

12.11 Mobile security

- Tokens in Keychain/Keystore; no secrets in the app bundle (assume the bundle can be reverse-engineered).
- TLS with optional certificate pinning (plan for rotation and backup pins).
- Encrypted local storage for the minimal offline cache and queue; wipe on logout, organization switch and after repeated failed unlock.
- Biometric gate and idle auto-lock; screenshot/screen-record protection for performance screens (configurable).
- Validate deep links and push payload targets; never perform actions directly from a deep link without authentication and permission checks.
- Obfuscation and release signing; remote kill-switch (minimum supported version) for urgent security fixes.

12.12 Infrastructure and cloud security

- Network: private subnets for compute and data; public access only via CDN/WAF/load balancer; security groups default-deny; no direct database access from the internet; administrative access through SSO-protected bastion or session manager with logging.
- Identity: separate cloud accounts per environment; least-privilege IAM roles for services and CI; no long-lived access keys; MFA on all human cloud accounts.
- Hardening: minimal container images, non-root users, read-only filesystems where possible, vulnerability scanning of images, automatic patching windows.
- Infrastructure as code with peer review and policy-as-code checks (for example public bucket detection).
- Availability: multi-AZ database, automated backups and point-in-time recovery, tested restore, documented RTO/RPO targets (to be set with the client), DDoS protection and graceful degradation.

12.13 Monitoring, detection and incident response

- Alert on: spike in failed logins, token reuse detection, permission changes outside business hours, mass exports, repeated authorization denials, webhook signature failures, abnormal API volume, new admin creation, integration secret access.
- Centralised, structured logs with request and tenant IDs; PII and secrets redaction in the logging layer; retain logs per policy; restrict log access.
- Incident response runbook: detect, triage severity, contain (revoke sessions/keys, isolate), eradicate, recover, notify affected customers and authorities as law requires, post-incident review with action items. Maintain an on-call contact list and communication templates.

Severity	Example	Initial response target (suggested)
SEV-1	Confirmed cross-tenant data exposure or credential compromise	Immediate; incident lead assigned; containment within hours; customer notification per contract/law
SEV-2	Exploitable vulnerability without evidence of abuse; major integration failure	Within 4 hours; fix or mitigate within 72 hours
SEV-3	Limited-impact bug or misconfiguration	Within 1 business day; fix in normal cycle

12.14 Privacy and compliance

- Treat the product as processing employees' personal data. Confirm applicable data-protection laws with the client (for example GDPR, India's DPDP Act 2023, local labour regulations on employee monitoring) and obtain legal review of the performance module before launch.
- Provide: privacy notice inside the product, configurable retention, data export (access request), correction and deletion workflows, consent or lawful-basis records where required.
- Maintain a data-processing inventory and subprocessor list (cloud, email, push, monitoring). Data residency options to be decided with the client.
- Transparency toward employees about what is collected, how scores are produced and who can see them is a product requirement, not only a legal one.
- Consider SOC 2 / ISO 27001 readiness if the platform will be sold to external companies; design evidence collection (audit logs, access reviews, change records) from day one.

12.15 Secure development lifecycle

Practice	When	Tooling examples
Threat modelling per major feature (auth, GitHub, performance, files, automation)	Phase 0 and before each module	STRIDE worksheet in ADR
Peer review with security checklist	Every PR	PR template
SAST and linting security rules	Every PR	CodeQL, ESLint security plugins, Semgrep
Dependency and license scanning	Every PR + weekly	Dependabot/Renovate, npm audit, OSV
Secret scanning	Every PR + repo history	GitHub secret scanning, gitleaks
Container and IaC scanning	Every build	Trivy, Checkov/tfsec
DAST on staging	Before each milestone release	OWASP ZAP
Tenant-isolation and permission regression tests	Every build	Custom integration suite
Independent penetration test	Before production launch and annually	External specialist
Access review of cloud, GitHub and admin accounts	Quarterly	Checklist

12.16 OWASP Top 10 coverage map

OWASP category	Where addressed
A01 Broken access control	3.4, 12.3, 12.4, tenant tests
A02 Cryptographic failures	12.5, 12.6
A03 Injection	12.7 (ORM, validation, allow-lists)
A04 Insecure design	12.1 threat model, workflow validators, idempotency, small-cohort suppression
A05 Security misconfiguration	12.7 headers/CORS, 12.12 infrastructure, IaC scanning
A06 Vulnerable and outdated components	12.15 dependency and container scanning
A07 Identification and authentication failures	12.2
A08 Software and data integrity failures	12.10 audit integrity, signed builds, pinned CI actions, webhook signatures
A09 Security logging and monitoring failures	12.10, 12.13
A10 Server-side request forgery	12.7 SSRF controls, automation allow-list

Category names follow the OWASP Top 10 (2021). Re-check against the latest published edition when the project starts.

12.17 Security acceptance checklist

#	Check	Phase
1	Password hashing, MFA, lockout, session rotation and revocation work as specified	1
2	Every endpoint has an explicit permission annotation; unannotated routes fail closed	1
3	Cross-tenant test suite passes for API, search, files, WebSocket and exports	1+
4	RLS enabled and forced on all tenant tables; app DB role is non-owner	1
5	Audit log records all sensitive actions listed in 12.10 and is append-only	1-2
6	Webhook HMAC verification and delivery dedupe tested with forged and replayed requests	3
7	GitHub tokens/private key encrypted, never logged or returned by API	3
8	File upload quarantine and AV scan flow works; unscanned files cannot be downloaded	1
9	Rate limits active on auth, search, export endpoints	1
10	Security headers, CSP and CORS allow-list verified on staging	1
11	Performance module: scope checks, access auditing, second-approver adjustments, small-cohort suppression verified	6
12	Mobile: secure storage, cache wipe, deep-link validation, minimum-version enforcement verified	7
13	Penetration test findings of high or critical severity resolved before launch	Launch
14	Backup restore test and incident-response drill completed	Launch

13. Data Model

The entity groups below come from the source specification. Column lists are Recommended starting points; finalize them in Phase 0 together with the permission matrix and metric definitions.

13.1 Modelling rules

- Every tenant-owned table carries **organization_id** directly (preferred) so RLS and indexes are simple. Primary keys are UUIDs (v7 preferred for index locality). Human-readable keys (DTH-123) are separate columns with a unique (project_id, number) constraint.
- Timestamps: created_at, updated_at (UTC). Soft delete with deleted_at where history matters; never hard-delete work, audit or performance data outside retention jobs.
- History and audit tables are append-oriented. Use a version column on mutable records for optimistic concurrency.
- GitHub data stores provider IDs, delivery IDs, provider timestamps and ingestion timestamps for deduplication and audit.
- Performance data stores period snapshots with model version and inputs so history is reproducible.
- Permissions are explicit (action x resource x scope). Do not rely on front-end hiding.
- Integration secrets are encrypted at rest and never selected into ordinary API responses.

13.2 Core tables and key columns

Table	Key columns (besides id, organization_id, timestamps)	Notes
organizations	name, slug, domain, status, settings (jsonb), retention_policy	Tenant root
users	email (citext unique), name, password_hash, mfa_enabled, status, timezone, locale	Global identity; membership per org
organization_members	user_id, status, primary_department_id, title, reports_to_user_id, joined_at	Reporting line drives manager scope
sessions	user_id, refresh_hash, family_id, device, ip, last_used_at, revoked_at	Rotation + reuse detection
invitations	email, token_hash, role_id, scope grants, expires_at, accepted_at, revoked_at	
departments / teams / team_members	name, head_id / lead_id, department_id, archived_at; team_id, user_id, role, from, to	Date-ranged membership for history
roles / permissions / role_permissions	role: name, is_system; permission: key (resource:action)	Keys defined in code
scope_assignments	user_id, role_id, scope_type, scope_id, granted_by, expires_at	Core of authorization
audit_logs	actor_type, actor_id, action, resource_type, resource_id, before, after, ip, request_id, prev_hash, hash	Insert-only
projects / project_members / project_teams	key, name, status, visibility, lead_id, health, start/end; role; team mapping	key unique per org
work_items	project_id, number, type, summary, description, status, priority, assignee_id, reporter_id, parent_id, sprint_id, estimate, rank, due_date, start_date, resolution, is_blocked, blocked_reason, version	Central record
work_item_history	work_item_id, actor_id, field, old_value, new_value, changed_at	Append-only
comments / attachments / labels / dependencies	work_item_id, author_id, body, visibility; storage_key, scan_status; type (blocks, relates, duplicates)	Attachment visible only if scan_status = clean
boards / board_columns / board_filters	project_id or team_id, type (kanban/scrum), status mapping, wip_limit	
sprints / sprint_items / sprint_events	team_id, name, goal, start/end, status, committed_snapshot, completed_snapshot; added_at, removed_at, reason	Snapshots power reports
milestones / releases / deployments / environments	project_id, due_date; version, status, approvals; environment_id, commit_sha, status, started/finished	
bugs (or work type), test_cases / test_runs / test_results	severity, environment, affected_version; steps, automation flag; run build, result, evidence	
documents / document_versions	title, owner_id, acl, status; version_no, content	
notifications / notification_preferences	recipient_id, event_type, target, read_at, delivered_status; channel settings	
worklogs / calendars / availability / capacity_allocations	work_item_id, user_id, minutes, date; working days; leave; sprint capacity	
github_accounts / github_connections / github_installations	user_id, github_user_id, token_enc; installation_id, status, permissions; repo grants	Tokens encrypted
repositories / repository_mappings	github_repo_id, full_name, visibility, default_branch, archived; project_id, team_id	Unique mapping rule

Table	Key columns (besides id, organization_id, timestamps)	Notes
github_events / commits / pull_requests / reviews / workflow_runs	delivery_id UNIQUE, event_type, payload (restricted), provider_ts, ingested_ts, processed_at; sha, author_identity; state, draft, merged_at; state, submitted_at; conclusion	Dedup + reconcile
github_identity_mappings	user_id, github_user_id, verified_via, verified_at, status	Only verified rows used for scoring
metric_definitions / analytics_snapshots	key, version, formula_ref; scope_type, scope_id, period, values (jsonb), computed_at	Immutable snapshots
dashboards / reports	owner_id, layout (jsonb), filters, schedule	
performance_periods / performance_weights	start, end, status; role_profile, category, weight, model_version	Weights versioned
performance_scores / performance_score_details	user_id, period_id, role_profile, overall, band, completeness, model_version; category, metric_key, raw, normalized, n, evidence_ids	Snapshot per period
performance_rankings / ranking_visibility_settings	scope, period, rank, score_band, eligible_count; level, mode (private/team/company), enabled_by	
performance_goals / reviews / feedback / badges	goal fields; review cycle; feedback visibility, acknowledged_at; badge definitions	
score_adjustments / score_disputes	score_id, author_id, reason, status, approver_id; evidence_id, employee_note, resolution	Two-person approval
automation_rules / automation_executions	scope, trigger, conditions, actions, version, enabled; rule_id, origin_chain_id, result, error	
integrations / api_keys / webhooks / webhook_deliveries	type, config_enc, status; key_hash, scopes, expires_at; url, secret_enc, events; response_code, attempt	Secrets encrypted/hashed

13.3 Example DDL (work_items) and indexing

```
CREATE TABLE work_items (
  id               uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id  uuid NOT NULL REFERENCES organizations(id),
  project_id       uuid NOT NULL REFERENCES projects(id),
  number           integer NOT NULL,                        -- DTH-<number>
  type             text NOT NULL,                           -- task | story | bug | epic | subtask
  summary          text NOT NULL CHECK (char_length(summary) BETWEEN 1 AND 255),
  description      text,
  status_id        uuid NOT NULL REFERENCES workflow_statuses(id),
  priority         smallint NOT NULL DEFAULT 3,
  assignee_id      uuid REFERENCES users(id),
  reporter_id      uuid NOT NULL REFERENCES users(id),
  parent_id        uuid REFERENCES work_items(id),
  sprint_id        uuid REFERENCES sprints(id),
  estimate_points  numeric(6,2),
  rank             text NOT NULL,                           -- lexicographic ordering key
  due_date         date,
  is_blocked       boolean NOT NULL DEFAULT false,
  version          integer NOT NULL DEFAULT 1,              -- optimistic concurrency
  created_at       timestamptz NOT NULL DEFAULT now(),
  updated_at       timestamptz NOT NULL DEFAULT now(),
  deleted_at       timestamptz,
  UNIQUE (project_id, number)
);
CREATE INDEX ON work_items (organization_id, project_id, status_id);
CREATE INDEX ON work_items (organization_id, assignee_id) WHERE deleted_at IS NULL;
CREATE INDEX ON work_items (organization_id, sprint_id, rank);
-- full-text: add generated tsvector column + GIN index; trigram index on summary for fuzzy match
```

13.4 Key relationships

- Organization 1-n Departments 1-n Teams; Teams n-n Users (team_members); Teams n-n Projects (project_teams).

- Project 1-n Work items; Work item 1-n Comments, Attachments, History, Worklogs; Work item n-n Work item (dependencies); Work item n-n Pull requests (links).
- Team 1-n Sprints; Sprint n-n Work items (sprint_items) with added/removed timestamps.
- Repository n-n Project (mapping); Repository 1-n Commits, Pull requests, Workflow runs; Pull request 1-n Reviews.
- User 1-n Performance scores (per period) 1-n Score details 1-n Evidence references (work item, PR, review, defect); Score 1-n Adjustments and Disputes.

14. API Specification Guide

14.1 Conventions

Topic	Convention
Style and versioning	REST over HTTPS, JSON. Base path /api/v1. Breaking changes require /v2. OpenAPI 3 document generated from code and published for the client generator.
Authentication	Authorization: Bearer access token (web mobile) or API key header for integrations. Tenant comes from the token, never from the URL or body.
Pagination	Cursor-based: ?limit=50&cursor;=... returns {data, next_cursor}. Max limit 100.
Filtering and sorting	Allow-listed fields only: ?status=in_progress&assignee;=me&sort;=-updated_at. Complex search through query objects validated by schema.
Errors	RFC 9457 problem-details style JSON with type, title, status, detail, code, request_id and field errors. No internals.
Idempotency	Idempotency-Key header on POST that creates resources or triggers actions; stored for 24 hours.
Concurrency	Resources return version/ETag; updates send If-Match or version; stale returns 409 with current state.
Rate limiting	Headers: X-RateLimit-Limit, -Remaining, -Reset; 429 with Retry-After.
Dates and IDs	ISO 8601 UTC timestamps; UUID ids plus human keys (DTH-123) accepted where documented.
Async operations	Exports and large reports return 202 with a job resource; poll or notify when ready; download link is signed and short-lived.
Deprecation	Sunset and Deprecation headers; minimum notice period for external API consumers.

```
HTTP/1.1 422 Unprocessable Content
Content-Type: application/problem+json
{
  "type": "https://api.devteamhub.example/problems/validation",
  "title": "Validation failed",
  "status": 422,
  "code": "VALIDATION_ERROR",
  "request_id": "req_01HV...",
  "errors": [ { "field": "dueDate", "message": "Due date cannot be before start date" } ]
}
```

14.2 Endpoint catalog (representative)

Area	Method and path	Purpose	Permission key
Auth	POST /auth/login | /auth/mfa/verify | /auth/refresh | /auth/logout	Sign in, MFA, rotate, sign out	public / session
Me	GET /me; PATCH /me/profile	Profile, memberships, effective permissions	self
Organizations	GET /organizations; POST /organizations/:id/select	List and switch tenant	membership
Users	GET/POST /users; PATCH /users/:id; POST /users/:id/deactivate	Lifecycle	user:manage
Invitations	POST /invitations; POST /invitations/:token/accept	Invite and join	user:invite / token
Roles	GET/POST /roles; POST /role-assignments	Role and scope admin	role:manage
Departments / Teams	GET/POST /departments; GET/POST /teams; POST /teams/:id/members	Structure	department:*, team:*
Projects	GET/POST /projects; GET/PATCH /projects/:id; /members; /milestones	Project management	project:*
Work items	GET/POST /work-items; GET/PATCH /work-items/:id; POST /work-items/:id/transitions	Core CRUD and workflow	work_item:*
Work item parts	POST /work-items/:id/comments | /attachments | /worklogs | /links; PATCH /work-items/:id/rank	Collaboration	work_item:comment etc.
Boards / Backlog	GET /boards/:id; GET /projects/:id/backlog	Board and backlog views	board:read
Sprints	POST /sprints; POST /sprints/:id/start | /complete; GET /sprints/:id/report	Sprint lifecycle	sprint:manage
Bugs / QA	GET/POST /bugs; /test-cases; /test-runs; PATCH /test-runs/:id/results	Quality	bug:*, test:*
Releases / Deployments	GET/POST /releases; POST /releases/:id/approve; GET/POST /deployments; POST /deployments/webhook	Release management	release:*, deployment:*
Documents	GET/POST /documents; GET /documents/:id/versions	Knowledge base	document:*
Search	GET /search?q=	Global search with ACL filtering	authenticated
GitHub	POST /github/connections; GET /github/repositories; POST /github/webhooks (public, signed); POST /github/identity/verify	Integration	integration:github:*
Analytics	GET /analytics/organization | /departments/:id | /teams/:id | /projects/:id; GET /reports/:id; POST /reports/:id/export	Reports	analytics:read:*, report:export
Performance	GET /performance/me | /performance/users/:id | /performance/leaderboard; POST /performance/feedback | /disputes | /adjustments	Insights	performance:read:*, performance:adjust
Notifications	GET /notifications; PATCH /notifications/:id; GET/PUT /notification-preferences	Inbox	self
Admin	GET/PATCH /settings; GET /audit-logs; POST /automation-rules; GET/POST /api-keys; /webhooks	Administration	org:admin, audit:read

Implementation note: the endpoint catalog is a starting contract. Each endpoint must be added to the OpenAPI document with request/response schemas, error codes and the permission key before implementation.

15. DevOps, Observability and Operations

15.1 CI/CD pipeline

Stage	Actions	Gate
Pull request	Install (locked), lint, type-check, unit tests, integration tests with ephemeral Postgres/Redis, OpenAPI diff, SAST, dependency + secret scan	All green and reviewed
Build	Build web, API, worker images; vulnerability scan; generate SBOM; sign artifacts	No critical vulnerabilities
Deploy dev	Automatic on merge to main; run migrations; smoke tests	Smoke tests pass
Deploy staging	Automatic or on tag; end-to-end tests (Playwright), permission and tenant suite, DAST scan, performance smoke	Suites pass
Release production	Manual approval; backward-compatible migration (expand-contract); blue/green or rolling deploy; automated rollback on failed health checks	Approval + health checks
Mobile	EAS/Fastlane builds per PR (internal), TestFlight/Play internal tracks, staged production rollout	QA sign-off

15.2 Database migrations and feature flags

- Use **expand-contract**: add new columns/tables first, deploy code that writes both, backfill, switch reads, then remove old structures in a later release. Never ship a migration that breaks the currently deployed version.
- Large backfills run as background jobs, not inside migration transactions.
- Ship unfinished modules behind feature flags (organization-level flags allow pilots, for example performance rankings).
- Every migration is reviewed for locks (use CONCURRENTLY for indexes) and tested on a production-sized copy.

15.3 Observability

- **Logs:** structured JSON with request_id, organization_id, user_id, route, latency; secrets and personal content redacted.
- **Metrics:** API latency (p50/p95/p99), error rate, DB pool and slow queries, queue depth and job age, webhook lag, sync lag per repository, WebSocket connections, notification delivery rate.
- **Traces:** OpenTelemetry across web, API, worker and external calls.
- **Dashboards:** service health, GitHub integration health, security events, tenant-level usage.
- **Alerts:** actionable and owned; page only on user-impacting or security issues; each alert links to a runbook.
- **Admin visibility:** background sync lag and webhook health shown to admins in the GitHub hub (Source requirement).

15.4 Reliability, backup and disaster recovery

- Health checks (liveness, readiness), graceful shutdown, queue retries with dead-letter queues, circuit breakers and timeouts on external calls.
- Multi-AZ PostgreSQL with point-in-time recovery; daily snapshots; cross-region backup copy if required; quarterly restore test.
- Agree RTO and RPO with the client (suggested starting point: RPO 15 minutes, RTO 4 hours) and document the DR plan.
- Runbooks: deploy/rollback, database restore, queue backlog, webhook failure, GitHub outage, key rotation, security incident, tenant data export/deletion.

16. Quality Assurance and Testing Strategy

Test type	Scope	Tooling (examples)	When
Unit	Domain rules: transition validators, capacity calculation, scoring math, permission evaluation	Jest/Vitest	Every PR
Integration	API + real PostgreSQL/Redis: CRUD, RLS, queues, GitHub webhook processing with fixture payloads	Jest + Testcontainers	Every PR
Contract	OpenAPI conformance; generated client compiles; GitHub payload fixtures versioned	Schema diff tools	Every PR
Permission matrix	For each role x endpoint: allowed and denied cases; field-level restrictions	Data-driven test suite	Every build
Tenant isolation	Cross-tenant attempts on all data paths	Custom suite	Every build
End-to-end (web)	Critical journeys: sign in, create item, move on board, plan and complete sprint, connect repo, view dashboard	Playwright	Staging each merge/nightly
Mobile	Sign in, offline queue, push deep link, approvals	Detox/Maestro + device farm	Before release
Performance	Large tenant dataset (for example 1M work items, 20M events); board, search, dashboard latency; queue throughput	k6	Before milestone
Security	SAST, DAST, dependency, container, auth and session abuse tests, pen test	See 12.15	Continuous + pre-launch
Accessibility	Keyboard paths, screen-reader labels, contrast	axe, manual audit	Each UI milestone
Fairness validation (performance)	Score distributions by cohort, sensitivity to weights, outlier review, replay on historical data	Notebook + review with HR	Phase 6 pilot

16.1 Test data and environments

- Seed scripts generate realistic multi-tenant data (several organizations, many roles, GitHub fixtures) for local and staging.
- Never copy production personal data to lower environments; if needed, anonymize with a reviewed script.
- Keep deterministic fixtures for scoring: a known set of events must always produce the same score and band.

16.2 Quality gates for a milestone

- No open critical or high defects; medium defects triaged with owners.
- Test coverage targets agreed (for example 80% on domain logic) and critical-path e2e suite green.
- Security checklist (12.17) items for the phase verified.
- Performance budget met on staging with representative data.
- Documentation, runbooks and API docs updated; demo accepted by product owner.

17. Delivery Plan, Acceptance and Open Decisions

17.1 Phases (Source) with indicative effort (Recommended)

Durations assume a core team of roughly 6 people (2 backend, 2 frontend, 1 mobile/full-stack, 1 QA) plus part-time design, DevOps and product owner. They are planning estimates to be re-baselined at the end of Phase 0 once scale, hosting budget and decisions are known.

Phase	Deliverables	Exit condition (Source)	Indicative weeks
0. Product foundation	UX flows, permission matrix, data model, architecture, design system, metric definitions, threat model, CI skeleton	Approved workflows, roles and MVP scope	3
1. Core workspace	Auth, organizations, members, departments, teams, projects, work items, comments, attachments, audit log basics, notifications foundation	Company can manage basic delivery work	6
2. Agile delivery	Boards, backlog, sprints, workflows, calendar, worklogs, basic dashboard	Teams can plan and track iterative work	6
3. GitHub integration	OAuth, GitHub App, repo mapping, PR/commit/review sync, webhook reconciliation, development panel, identity mapping	Engineering events link to delivery work	5
4. Quality and release	Bugs, test management, releases, deployments, traceability	Teams can track quality and release readiness	5
5. Analytics	Dashboards, project/team/org reports, snapshots, exports	Leads can inspect delivery and quality trends	5
6. Performance insights	Role-based metrics, evidence, private self-view, configurable leaderboard, dispute flow, pilot	Transparent, fair score pilot validated	8 (incl. pilot)
7. Mobile and scale	React Native app, push, offline-safe updates, hardening, penetration test, launch readiness	Mobile workflows and operational readiness	6

MVP recommendation (Source)
Strongest MVP = organization + team + project + work item + board + sprint + GitHub connection + basic delivery dashboard + notifications. Build Phases 0-3 plus the basic dashboard and notifications first (about 5 months with the team above). Add quality/release, detailed analytics and performance rankings only after work and integration data are reliable. Performance rankings launch only after a pilot validates fairness and trust. A minimal mobile experience (My Work, notifications, comment, status update) may be pulled forward after the MVP if the client prefers.

17.2 Milestone acceptance checklist

Milestone	Must be demonstrable on staging	Evidence to hand over
M1 Foundation (end Phase 0)	Clickable flows, approved permission matrix, ERD, architecture and threat model, running CI	Signed-off documents, ADRs, backlog for Phase 1
M2 Core workspace (Phase 1)	Invite user, assign role/scope, create department/team/project, create and edit work items with history and comments, attach file with scan, view audit log	Test report, tenant-isolation results, API docs
M3 Agile (Phase 2)	Configure board, manage backlog, plan/start/complete sprint, burndown, worklog, calendar, basic dashboard, notifications	E2E recordings, performance results
M4 MVP complete (Phase 3)	Install GitHub App, select repos, map to project, PR links to work item, webhook replay and reconciliation, identity mapping, development panel	Webhook test report, GitHub health dashboard, security checklist items 1-10
M5 Quality and release (Phase 4)	Bug workflow, test run with defect creation, release with readiness checklist and approval, deployment webhook	Traceability report
M6 Analytics (Phase 5)	Team/project/org analytics with drill-down, scope enforcement, snapshots, async export	Metric definitions document, scope test results
M7 Performance pilot (Phase 6)	Private self-view with evidence, dispute flow, adjustment approval, leaderboard toggle, fairness report	Pilot feedback, fairness analysis, legal/HR review note
M8 Launch (Phase 7)	Mobile app in stores/TestFlight, push, offline queue, load test, DR drill, penetration test fixed	Runbooks, pen test report, launch checklist

17.3 Cross-module acceptance criteria (Source)

- A user can join an organization, receive a role and access only resources allowed by that role and scope.
- An admin can create departments, teams, projects, members and assign team/project access.
- A permitted user can create, assign, update, transition, comment on and track a work item with complete history.
- A team can configure a board, manage a backlog, plan/start/complete a sprint and inspect sprint outcomes.
- A user can connect a personal GitHub account; an admin can install the GitHub App and select repositories.
- Repository, PR, commit, review and deployment activity can be synchronized, deduplicated and linked to work items.
- Dashboards and analytics are visible only within access scope; exports enforce the same scope.
- Performance scores show role-specific methodology, evidence, completeness and historical snapshot; users can challenge inaccurate evidence.
- Admins can configure individual/team/company ranking visibility; private by default.
- Sensitive admin, integration, permission, export and score-adjustment actions are auditable.
- Mobile users can securely view and update work, comment, receive notifications and create basic tasks.

17.4 Risks and mitigations

Risk	Impact	Mitigation
Permission model complexity causes leaks or confusing behaviour	High	Define matrix in Phase 0; central authorization service; matrix-driven tests; effective-permission preview
GitHub rate limits, webhook loss, event ordering	Medium	Queue + idempotency + reconciliation; conditional requests; visible sync health
Performance score seen as unfair or used punitively	High	Pilot, explainability, dispute flow, private default, legal/HR review, no automatic decisions
Scope creep (trying to build all Jira features)	High	MVP discipline, change control, phase gates
Large-tenant performance problems (boards, search, analytics)	Medium	Indexes, pagination, snapshots, load tests with realistic data from Phase 2
Mobile offline conflicts	Medium	Limit offline scope, idempotent queue, conflict UI
Legal/privacy constraints on employee monitoring	High	Early legal review, data minimisation, transparency, configurable retention
Single-person dependency (freelancer bus factor)	Medium	ADRs, documentation, infrastructure as code, regular hand-over, repository access for client

17.5 Open decisions with recommended defaults

Raise each item with the product owner before building the affected module. The recommended default lets work proceed if no answer is available, but must be logged as an assumption.

Decision (Source list)	Recommended default	Needed by
Hierarchy: multiple departments/teams per user?	Many teams per user; one primary department	Phase 0
Role matrix, custom roles, project visibility, external collaborators	Use presets in Section 3.3; custom roles in Phase 1b; external collaborators as restricted Viewer with explicit project grants and expiry	Phase 0
Work types, workflow templates, estimation, sprint conventions	Epic, Story, Task, Bug, Sub-task; Fibonacci story points (hours optional); two-week sprints; templates from Section 5	Phase 1
GitHub scopes, repository selection, retention, identity matching	Permissions in Section 8.1; explicit repo selection; raw event payload retention 90 days, normalized data per org policy; OAuth-verified identity	Phase 3
Analytics definitions, windows, snapshot frequency, export permissions	Definitions in Section 9.1; windows: day/week/sprint/month/quarter; nightly + sprint-event snapshots; export separate permission	Phase 5
Performance weights by role, eligibility, privacy defaults, dispute process, who can adjust	Section 9.5 weights; 60% active days and 70% completeness; private default; dispute SLA 5 working days; adjustments need two authorized approvers	Phase 6
Notification channels, retention, file limits, mobile offline scope, SSO providers	In-app, email, push; notifications 12 months; 25 MB per file; offline = My Work + recently viewed with queued status/comments; SSO Google and Microsoft first	Phase 1-2
Organization size, concurrency, repository count, event volume, hosting budget	Assume up to about 2,000 users per tenant and tens of thousands of events per day for sizing until confirmed	Phase 0

Appendices

Appendix A - Source specification coverage map

Use this table to confirm that every page in the original specification is covered by a section in this document.

Source section	Covered in
3.1 Sign in; 3.2 Accept invitation; 3.3 Organization selection	6.1, 6.2, 6.3
3.4 Home dashboard; 3.5 Global search; 3.6 My Work	6.4, 6.5, 6.6
3.7 Work item detail; 3.8 Create/edit modal	6.7, 6.8
3.9 Project directory; 3.10 Project overview; 3.11 Project settings	6.9, 6.10
3.12 Team; 3.13 Department; 3.14 People and profile	6.11, 6.12
3.15 Board; 3.16 Backlog; 3.17-3.19 Sprint planning, active sprint, report	7.1, 7.2, 7.3
3.20 Calendar and timeline	6.13
4.1 Bugs; 4.2 QA; 4.3 Releases; 4.4 Deployments	7.4, 7.5, 7.6
4.5 Work log; 4.6 Knowledge base; 4.7 Comments	7.7, 7.8, 7.9
5.1-5.5 GitHub hub, repositories, PR panel, activity, behaviour rules	8.1 - 8.5
6.1-6.5 Analytics landing, organization, department, team, project	9.1, 9.2, 9.3
6.6-6.9 Individual performance, leaderboard, methodology, goals; scoring model	9.4 - 9.8
7.1-7.8 Admin: org settings, users, roles, workflows, automation, notifications, integrations, audit	10.1 - 10.8
8 Mobile pages and requirements	11.1 - 11.7
9 Shared interaction, validation, notifications	11.8
10 Data model and API surface	13, 14
11 Architecture and NFRs	4, 12, 15, 16
12 Acceptance criteria, roadmap, MVP, decisions	17

Appendix B - Pull request checklist

- Linked work item key in branch and PR title; description explains why, not only what.
- Screenshots or recordings for UI changes, including empty, error and denied states.
- New endpoints have permission annotation, DTO validation, OpenAPI entry, audit event if sensitive.
- New tables have organization_id, RLS policy, indexes and migration notes.
- Tenant-isolation and permission tests added or updated.
- No secrets, tokens or personal data in code, logs or test fixtures.
- Performance considered: pagination, indexes, no N+1.
- Accessibility checked (keyboard, labels, contrast).
- Rollback plan and feature flag noted when risky.

Appendix C - Environment variables (names only)

```
APP_ENV, APP_BASE_URL, API_BASE_URL, ALLOWED_ORIGINS
DATABASE_URL (app role), DATABASE_MIGRATION_URL (migration role), REDIS_URL
JWT_PRIVATE_KEY_REF, JWT_PUBLIC_KEYS, ACCESS_TOKEN_TTL, REFRESH_TOKEN_TTL
PASSWORD_PEPPER_REF, ENCRYPTION_KEY_ID (KMS), COOKIE_DOMAIN
GITHUB_APP_ID, GITHUB_APP_PRIVATE_KEY_REF, GITHUB_APP_CLIENT_ID, GITHUB_APP_CLIENT_SECRET_REF
GITHUB_WEBHOOK_SECRET_REF
OIDC_GOOGLE_CLIENT_ID/SECRET_REF, OIDC_MICROSOFT_CLIENT_ID/SECRET_REF
S3_BUCKET_CLEAN, S3_BUCKET_QUARANTINE, S3_REGION, AV_SCAN_ENDPOINT
EMAIL_PROVIDER_KEY_REF, PUSH_FCM_CREDENTIALS_REF, APNS_KEY_REF
SENTRY_DSN, OTEL_EXPORTER_ENDPOINT, LOG_LEVEL
FEATURE_FLAGS_SOURCE
```

Values are never committed. *_REF variables point to entries in the secrets manager.

Appendix D - Glossary of engineering terms

Term	Meaning
BOLA / IDOR	Broken object-level authorization / insecure direct object reference: accessing another user's object by changing an ID.
Outbox pattern	Writing events to a database table in the same transaction as the change, then publishing them reliably.
Idempotency	Repeating the same request has the same effect as doing it once.
Expand-contract	Migration technique that adds new structures first and removes old ones later for zero-downtime deployments.
Envelope encryption	Encrypting data with a data key, then encrypting that key with a master key held in KMS.
HMAC	Keyed hash used to verify that a message came from someone with the shared secret.
Shrinkage	Statistical technique pulling small-sample values toward the group median to reduce noise.
DORA metrics	Deployment frequency, lead time for changes, change failure rate and time to restore service.
ADR	Architecture decision record: short document capturing a decision and its reasoning.

End of document
This blueprint is a proposed implementation guide derived from the DevTeam Hub product specification of 1 November 2026. Final requirements, scoring weights, privacy defaults and hosting decisions must be confirmed with representative employees, team leads, administrators and company stakeholders before the affected modules are built.